Proceedings of

SAT COMPETITION 2025

Solver and Benchmark Descriptions

Cayden Codel, Katalin Fazekas, Marijn J. H. Heule, Markus Iser *(editors)*

Proceedings of SAT Competitions –
Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions

Volume Editors:
Cayden Codel
Katalin Fazekas
Marijn J. H. Heule
Markus Iser

DOI: 10.34726/10379
Vienna 2025

This work is licensed under a Creative Commons "Attribution 4.0 International" license.

PREFACE

The area of Boolean satisfiability (SAT) solving keeps on making progress. Besides new algorithms and better heuristics, refined implementation techniques turned out to be vital for the success story of SAT solving. To keep up the driving force in improving SAT solvers, SAT solver competitions provide opportunities for solver developers to present their work to a broader audience and to objectively compare the performance of their own solvers with that of other state-of-the-art solvers.

SAT Competition 2025 (SC 2025, `https://satcompetition.github.io/2025/`), a competitive event for SAT solvers, was organized as a satellite event of the 28th International Conference on Theory and Applications of Satisfiability Testing (SAT 2025). SC 2025 stands in the tradition of the previously organized main competitive events for SAT solvers: the SAT Competitions held 2002–2005; biannually during 2007–2013; 2014, 2016–2018, and 2020–2024; the SAT Races held in 2006, 2008, 2010, 2015, and 2019; and SAT Challenge 2012.

SC 2025 consisted of a total of two tracks: Main Track (with No Limits sub-track), and Parallel Track. There were two ways of contributing to SC 2025: by submitting one or more solvers to participate in the competition; and by submitting interesting benchmark instances on which the submitted solvers could be evaluated in the competition.

The rules of SC 2025 required all contributors to submit a short, 1–2 page long description as part of their contribution. This compilation contains these non-peer-reviewed descriptions in a single volume, providing a way of consistently citing the individual descriptions and finding out more details on the individual solvers and benchmarks.

Successfully running SC 2025 would not have been possible without active support from the community at large. We would like to thank the Software and Computational Systems Lab (SoSy Lab) at Ludwig-Maximilians-Universität München (LMU) (https://www.sosy-lab.org/) for the computing resources needed to run SC 2025. Many thanks go to Dirk Beyer and Philipp Wendler for their invaluable help in setting up the BenchCloud instance to accommodate for the needs of the competition. Furthermore, we thank Amazon, in particular Robert Jones and Benjamin Jones, for providing the resources to support the development of parallel and distributed SAT solvers on the AWS cloud and for executing the Parallel track. Finally, we thank Tomáš Balyo and Filuta AI for sponsoring the awarded medals.

Finally, we would like to emphasize that a competition does not exist without participants: we thank all those who contributed to SC 2025 by submitting either solvers or benchmarks and the related descriptions.

Cayden Codel, Katalin Fazekas, Marijn J. H. Heule, Markus Iser,
SAT Competition 2025 Organizers

Contents

Preface 3

Contents **5**

Solver Descriptions

Satsuma+Kissat in the SAT Competition 2025
Markus Anders, Bart Bogaerts, Benjamin Bogø, Arthur Gontier, Wietze Koops, Jakob Nordström and Andy Oertel 8
CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2025
Armin Biere, Tobias Faller, Mathias Fleury, Nils Froleyks and Florian Pollitt . 10
Kissat-CURE: Enhancing SAT Solving Through Dynamic Configuration Tuning and Cold Restart Techniques
Yalun Cai, Mengxia Tao, Kezhi Li, Jiaying Zhu, Zhengyuan Shi and Qiang Xu 11
Kissat_CoRephase_CoReward and Kissat_MAB_CoRephase solvers in SAT Competition 2025
Shuang Chen, Jianmin Zhang, Jinjin Liu, Yan Sun and Tiejun Li 13
A Self-Optimizing Framework for SAT Solvers via Population Evolution and Large Language Model Collaboration
Hang Ding, Mao Luo, Chu-Min Li, Shunwei Li, Runyao Chen, Caiquan Xiong and Xinyun Wu 15
Kissat_VSA in the SAT Competition 2025
Shuolin Li and Jialu Zhang 18
Kissat_Scavel_RS and Kissat_Scavel2025
Zhihui Li, Guanfeng Wu, Li Chen, Shuwei Chen and Yang Xu 19
Kissat-Pred and Kissat-Pred-Aggressive in SAT Competition 2025
Chuan Luo, Zhenwei Yang, Shaoke Cui and Chunming Hu 20
PRS: A parallel framework for SAT
Yuhang Qian, Zhihan Chen, Xindi Zhang and Shaowei Cai 21
Reencoding Unique Literal Clauses in CaDiCaL
Joseph E. Reeves and Aeacus Sheng 22
Painless in SAT Competition 2025 with PL-PRS-KISSAT and PL-PRS-GASPI-KISSAT
Mazigh Saoudi, Souheib Baarir, Julien Sopena, Thibault Lejemble and Sabrine Saouli 24
MallobSat naps in the SAT Competition 2025
Dominik Schreiber, Niccolò Rigi-Luperti and Armin Biere 25

dilemma: A SAT-solver using a dilemma calculus on And-Inverter-Graphs
Benjamin Takacs . . . 26
hKis, hCaD, PRSPaKisInc and SBVAPRSPaKisInc in the SAT Competition 2025
Rodrigue Konan Tchinda and Clémentin Tayou Djamegni . . . 27
DynamicCaDiCaL
Junjie Xu . . . 28
Activity-Guided Dynamic BVA for amSAT
Chuang Zeng, Mao Luo, Chu-Min Li, Shuolin Li, Yiwen Huang and Zhipeng Lü 30

Benchmark Descriptions

Hard Combinatorial Instances with Symmetry
Markus Anders, Bart Bogaerts, Benjamin Bogø, Arthur Gontier, Wietze Koops, Jakob Nordström and Andy Oertel . . . 33
Split-BDD Multiplier Verification Benchmarks Submitted to the SAT Competition 2025
Armin Biere, Alexander Konrad, Daniela Kaufmann and Christoph Scholl . . . 34
Bit-Level and Global Logical Equivalence Checking of Hardware Multiplier Architectures
Yalun Cai, Mengxia Tao, Kezhi Li, Jiaying Zhu, Zhengyuan Shi and Qiang Xu 35
Benchmarks from equivalence checking of MD5 circuits
Shuang Chen, Jianmin Zhang and Yan Sun . . . 37
Challenging Certificates from Model Checking
Nils Froleyks, Emily Yu and Armin Biere . . . 38
SAT Encodings of P-Center Problem
Shuolin Li and Jialu Zhang . . . 39
Benchmarks Encoding the Preferred Extension of Argument Framework
Qi Liu, Mao Luo, Chu-Min Li and Caiquan Xiong . . . 40
Combinational Equivalence Checking Instances
Chuan Luo, Zhenwei Yang, Shaoke Cui and Chunming Hu . . . 42
Odd Ball Weighing Problems
Andrew Mihal . . . 43
Bel's Pyramid Puzzle
Andrew Mihal and Belgacem (Bel) Haba . . . 45
CNF Generation of Miter Circuits
Yuhang Qian, Zhihan Chen, Xindi Zhang and Shaowei Cai . . . 47
Round-Robin Tournament Scheduling Benchmarks
Joseph E. Reeves and Aeacus Sheng . . . 49
SAT Encodings of Mechanical Master-Key Systems
Niccolò Rigi-Luperti and Dominik Schreiber . . . 50
Benchmark Description for dyCaDiCaL Submission to SAT Competition 2025
Junjie Xu . . . 52

Indices

Solver Index . . . 53
Benchmark Index . . . 54
Author Index . . . 56

SOLVER DESCRIPTIONS

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Satsuma+Kissat in the SAT Competition 2025

Markus Anders
RPTU Kaiserslautern-Landau
Kaiserslautern, Germany
0009-0004-5992-8433

Bart Bogaerts
KU Leuven
Leuven, Belgium
0000-0003-3460-4251

Benjamin Bogø
University of Copenhagen
Copenhagen, Denmark
0009-0000-2192-3291

Arthur Gontier
Glasgow University
Glasgow, United Kingdom
0000-0002-9579-5584

Wietze Koops
Lund University
Lund, Sweden
0000-0001-9945-1992

Jakob Nordström
University of Copenhagen & Lund University
Copenhagen, Denmark & Lund, Sweden
0000-0002-2700-4285

Andy Oertel
Lund University
Lund, Sweden
0000-0001-9783-6768

***Abstract*—The configuration SATSUMA+KISSAT combines the static symmetry breaking algorithm SATSUMA with the SAT solver KISSAT, and produces proofs in the VERIPB format.**

I. Introduction

Symmetry handling is an important default technique used in a wide range of combinatorial solving paradigms, including mixed-integer programming [1] and constraint programming [2]. Together with powerful solvers, it is used to resolve long-standing mathematical problems [3] as well as real-world industrial instances [4].

The present solver configuration consists of the symmetry breaking tool SATSUMA [5] (version 1.3) and the SAT solver KISSAT [6] (version 4.0.2). The goal of SATSUMA is to handle symmetry as well as possible, while only incurring reasonable overhead.

The submission furthermore relies on recent improvements to the VERIPB proof system [7], [8] that enables more efficient proof logging for symmetry breaking constraints.

As such, the aim of this submission is to

1) demonstrate that symmetry breaking can be a beneficial default strategy in modern SAT solvers
2) *even* if it is required that formally verifiable certificates are produced and checked.

II. Techniques

The overall toolchain is quite simple and works as follows. First, the instance is passed to SATSUMA, which acts as a preprocessor and adds additional clauses to the formula. Then, the resulting instance is passed to KISSAT.

SATSUMA uses fixed internal limits on the number of certain operations, the size of the produced symmetry breaking predicate, and the size of graph components. These limits aim to ensure that the tool only runs for a reasonable amount of time, and the produced constraints do not overwhelm the solver or proof logging toolchain.

In the following, we give some more detailed information of the technical improvements used in the submission.

A. Symmetry Breaking Algorithm

We describe some technical improvements over previous symmetry breaking algorithms, in particular compared to BREAKID [9].

a) New Detection Algorithms: SATSUMA uses special symmetry detection algorithms for particular group structures. These detection algorithms are based on the individualization-refinement framework, and are used to detect *row symmetry*, *row-column symmetry*, and *Johnson symmetry* in the underlying instances. These are frequently occurring structures in some instance families. Row-column and Johnson symmetries are not detected as explicit structures by previous tools.

b) Efficiency Improvements: The tool furthermore makes various algorithmic changes over previous algorithms.

- SATSUMA is built around the DEJAVU [10], [11] library for symmetry detection, while BREAKID is built around the SAUCY [12] tool.
- The graph encoding used for CNF formulas is based on an efficient, classic encoding [13].
- The algorithms and data structures used to handle symmetry severely differ from BREAKID. For example, symmetry breaking constraints are constructed in linear-time in the size of the *support* of symmetries.

B. Proof Logging

The solver configuration outputs VERIPB proofs. We make use of a recent addition to the proof system which allows for more efficient proofs of symmetry breaking clauses. We use the dominance rule with a preorder definition that uses *auxiliary variables* to define the order, as is described in more detail the VERIPB documentation [14]. Roughly, this addition enables us to write a proof which is *linear* in the size of the symmetry breaking constraint, as opposed to the previous encoding which was *quadratic*.

Partially funded by the European Union (ERC, CertiFOX, 101122653), Fonds Wetenschappelijk Onderzoek – Vlaanderen (projects G070521N, G064925N), the Swedish Research Council grant 2016-00782, the Independent Research Fund Denmark grant 9040-00389B, and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

III. Availability

The code of the submission will be made available in the SATSUMA repository[1].

IV. Acknowledgement

We want to thank everyone who contributed to KISSAT for their efforts, and for making the tool publicly available.

Markus Anders thanks Sofia Brenner and Pascal Schweitzer for their help in creating the symmetry algorithms that make this submission possible.

We also want to thank all contributors to VeriPB, and in particular Stephan Gocht for his monumental contributions towards pseudo-Boolean proof logging.

References

[1] S. Bolusani, M. Besançon, K. Bestuzheva, A. Chmiela, J. Dionísio, T. Donkiewicz, J. van Doornmalen, L. Eifler, M. Ghannam, A. Gleixner, C. Graczyk, K. Halbig, I. Hedtke, A. Hoen, C. Hojny, R. van der Hulst, D. Kamp, T. Koch, K. Kofler, J. Lentz, J. Manns, G. Mexi, E. Mühmer, M. E. Pfetsch, F. Schlösser, F. Serrano, Y. Shinano, M. Turner, S. Vigerske, D. Weninger, and L. Xu, "The SCIP Optimization Suite 9.0," Optimization Online, Technical Report, February 2024, available at https://optimization-online.org/2024/02/the-scip-optimization-suite-9-0/.

[2] T. Walsh, "General symmetry breaking constraints," in *Proceedings of the 12th International Conference on Principles and Practice of Constraint Programming (CP '06)*, ser. Lecture Notes in Computer Science, vol. 4204. Springer, Sep. 2006, pp. 650–664.

[3] V. Angeltveit and B. D. McKay, "$R(5,5) \leq 46$," 2024. [Online]. Available: https://arxiv.org/abs/2409.15709

[4] P. Bendotti, P. Fouilhoux, and C. Rottner, "Orbitopal fixing for the full (sub-)orbitope and application to the unit commitment problem," *Mathematical Programming*, vol. 186, pp. 337–372, 2021.

[5] M. Anders, S. Brenner, and G. Rattan, "Satsuma: Structure-based symmetry breaking in SAT," in *Proceedings of the 27th International Conference on Theory and Applications of Satisfiability Testing (SAT '24)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 305, Aug. 2024, pp. 4:1–4:23.

[6] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024," in *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*, ser. Department of Computer Science Report Series B, vol. B-2024-1. University of Helsinki, 2024, pp. 8–10.

[7] B. Bogaerts, S. Gocht, C. McCreesh, and J. Nordström, "Certified dominance and symmetry breaking for combinatorial optimisation," *Journal of Artificial Intelligence Research*, vol. 77, pp. 1539–1589, Aug. 2023, preliminary version in *AAAI '22*.

[8] S. Gocht and J. Nordström, "Certifying parity reasoning efficiently using pseudo-Boolean proofs," in *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI '21)*, Feb. 2021, pp. 3768–3777.

[9] J. Devriendt, B. Bogaerts, M. Bruynooghe, and M. Denecker, "Improved static symmetry breaking for SAT," in *Proceedings of the 19th International Conference on Theory and Applications of Satisfiability Testing (SAT '16)*, ser. Lecture Notes in Computer Science, vol. 9710. Springer, Jul. 2016, pp. 104–122.

[10] M. Anders and P. Schweitzer, "Parallel computation of combinatorial symmetries," in *29th Annual European Symposium on Algorithms (ESA '21)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 204, 2021, pp. 6:1–6:18.

[11] M. Anders, "Efficient algorithms for symmetry detection," Ph.D. dissertation, TU Darmstadt, 2024.

[12] H. Katebi, K. A. Sakallah, and I. L. Markov, "Symmetry and satisfiability: An update," in *Proceedings of the 13th International Conference on Theory and Applications of Satisfiability Testing (SAT '10)*, ser. Lecture Notes in Computer Science, vol. 6175. Springer, Jul. 2010, pp. 113–127.

[13] F. A. Aloul, A. Ramani, I. L. Markov, and K. A. Sakallah, "Solving difficult instances of boolean satisfiability in the presence of symmetry," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 22, no. 9, pp. 1117–1137, 2003.

[14] M. Anders, B. Bogaerts, B. Bogø, A. Gontier, W. Koops, C. McCreesh, M. O. Myreen, J. Nordström, A. Oertel, A. Rebola-Pardo, and Y. K. Tan, "Documentation of VeriPB and CakePB for the SAT competition 2025," Apr. 2025.

[1] https://github.com/markusa4/satsuma

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2025

Armin Biere*, Tobias Faller*, Mathias Fleury*, Nils Froleyks‡, Florian Pollitt*
* *University of Freiburg*, Germany ‡ *Johannes Kepler University*, Linz, Austria

In essence the same versions of Gimsatul, IsaSAT and Kissat were submitted to the SAT Competition 2025 as those submitted in 2024 [1]. Note, that Kissat was dominating the competition in the main track in 2024. However it lacks many features supported by CaDiCaL. For instance Kissat does not support incremental solving. Thus, instead of trying to improve Kissat, we put our efforts into porting missing features from Kissat to CaDiCaL. This new submitted version of CaDiCaL is available on the development branch on Github. Its code base increased roughly from 46 KLOC to 64 KLOC of C++, needed to support the following techniques ported from Kissat:

- *clausal congruence closure* [2]
- *bounded variable addition* [3], [4]
- *ticks based scheduling* [5]
- *revisited vivification* [6]
- *look-ahead lucky phases* [1]
- *clausal equivalence sweeping* [7]
- *semantic definition mining* [8]

The latter two techniques rely on the internal sub-solver Kitten (from Kissat), which adds another roughly 3 KLOC of C.

On the practical side a major challenge was to produce LRAT proofs with antecedents for all the advanced inprocessing we added, particularly for clausal congruence closure. This is in contrast to Kissat, which only supports easier to produce, more compact but much slower to check DRUP proofs.

On the theoretical side, even though not important for the SAT competition, we had to come up with a new clausal proof calculus [9]. It allows to add specific blocked clauses during incremental SAT solving, including some form of extended resolution, necessary to produce proofs when using bounded variable addition in an incremental setting. Also the API changed slightly as the user now needs to ask the solver for new available variables indices instead of using them freely.

Preliminary results suggest that this improves run-time of stand-alone solving substantially, allows to solve benchmarks which require bounded variable addition or congruence closures, but still falls behind Kissat, unless we also take proof checking into account. The combined time of proof production and checking for CaDiCaL is much smaller than for Kissat.

This work was supported in part by the state of Baden-Württemberg through bwHPC, the German Research Foundation (DFG) through grant INST 35/1597-1 FUGG.

References

[1] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024," in *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*, ser. Department of Computer Science Report Series B, M. Heule, M. Iser, M. Järvisalo, and M. Suda, Eds., vol. B-2024-1. University of Helsinki, 2024, pp. 8–10.

[2] A. Biere, K. Fazekas, M. Fleury, and N. Froleyks, "Clausal congruence closure," in *27th International Conference on Theory and Applications of Satisfiability Testing, SAT 2024, August 21-24, 2024, Pune, India*, ser. LIPIcs, S. Chakraborty and J. R. Jiang, Eds., vol. 305. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, pp. 6:1–6:25.

[3] N. Manthey, M. Heule, and A. Biere, "Automated reencoding of boolean formulas," in *Hardware and Software: Verification and Testing - 8th International Haifa Verification Conference, HVC 2012, Haifa, Israel, November 6-8, 2012. Revised Selected Papers*, ser. Lecture Notes in Computer Science, A. Biere, A. Nahir, and T. E. J. Vos, Eds., vol. 7857. Springer, 2012, pp. 102–117.

[4] A. Haberlandt, H. Green, and M. J. H. Heule, "Effective auxiliary variables via structured reencoding," in *26th International Conference on Theory and Applications of Satisfiability Testing, SAT 2023, July 4-8, 2023, Alghero, Italy*, ser. LIPIcs, M. Mahajan and F. Slivovsky, Eds., vol. 271. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, pp. 11:1–11:19.

[5] A. Biere, "CaDiCaL, Lingeling, Plingeling, Treengeling, YalSAT Entering the SAT Competition 2017," in *Proc. of SAT Competition 2017 – Solver and Benchmark Descriptions*, ser. Department of Computer Science Series of Publications B, T. Balyo, M. J. H. Heule, and M. Järvisalo, Eds., vol. B-2017-1. University of Helsinki, 2017, pp. 14–15.

[6] F. Pollitt, M. Fleury, A. Biere, M. Heule, K. Sakallah, J. Chen, and Y. Fisseha, "Revisiting clause vivification," in *Joint Proceedings of the 23rd International Workshop on Satisfiability Modulo Theories (SMT'25) and of the 16th Pragmatics of SAT International Workshop (POS'25), co-located with the 28th International Conference on Theory and Applications of Satisfiability Testing (SAT'23), Glasgow, UK, July, 11, 2025*, ser. CEUR Workshop Proceedings, A. N. Jochen Hoenicke, Mikoláš Janota and S. Tourret, Eds. CEUR-WS.org, 2025, to be published.

[7] A. Biere, K. Fazekas, M. Fleury, and N. Froleyks, "Clausal equivalence sweeping," in *Proceedings 24th International Conference on Formal Methods in Computer-Aided Design (FMCAD'24)*, N. Narodytska and P. Rümmer, Eds. TU Wien Academic Press, 2024, pp. 236–241.

[8] M. Fleury and A. Biere, "Mining definitions in Kissat with Kittens," *Formal Methods Syst. Des.*, vol. 60, no. 3, pp. 381–404, 2022.

[9] K. Fazekas, F. Pollitt, M. Fleury, and A. Biere, "Incremental inprocessing rules beyond resolution," in *Joint Proceedings of the 23rd International Workshop on Satisfiability Modulo Theories (SMT'25) and of the 16th Pragmatics of SAT International Workshop (POS'25), co-located with the 28th International Conference on Theory and Applications of Satisfiability Testing (SAT'23), Glasgow, UK, July, 11, 2025*, ser. CEUR Workshop Proceedings, A. N. Jochen Hoenicke, Mikoláš Janota and S. Tourret, Eds. CEUR-WS.org, 2025, to be published.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Kissat-CURE: Enhancing SAT Solving Through Dynamic Configuration Tuning and Cold Restart Techniques

Yalun Cai*†, Mengxia Tao*, Kezhi Li*†, Jiaying Zhu*†, Zhengyuan Shi*†, Qiang Xu*†
*National Center of Technology Innovation for EDA, Nanjing, China
†Department of Computer Science and Engineering, The Chinese University of Hong Kong, Sha Tin, Hong Kong S.A.R.

***Abstract*—We present Kissat-CURE, a novel SAT solver that extends the state-of-the-art Kissat-4.0.2 framework with two complementary techniques: Dynamic Configuration Tuning (DCT) and Cold Restart (CR), designed to enhance solving efficiency across diverse problem instances. Kissat-CURE was submitted to the Main Track in SAT Competition 2025.**

I. Kissat

Kissat [1], widely recognized as one of the state-of-the-art SAT solvers and the champion of the SAT Competition 2024, has earned considerable acclaim within the SAT community for its sophisticated heuristic design and efficient implementation. All techniques presented in this work are implemented based on Kissat-4.0.2, the latest version of the Kissat framework.

II. Dynamic Configuration Tuning

State-of-the-art SAT solvers typically incorporate numerous configurations, while SAT instances exhibit considerable structural diversity. A configuration that performs optimally on one instance may yield suboptimal results on another. Furthermore, we observe that CNF formulations undergo significant transformations during the solving process. The evolving CNF instance progressively diverges from its original state due to critical clause operations, including clause elimination and clause learning. These phenomena underscore the necessity for dynamically adjusting configurations throughout the solving process to accommodate the evolving CNF structure. To address this problem, we model configuration tuning as a Multi-Armed Bandit (MAB) problem and present Dynamic Configuration Tuning (DCT), a novel MAB-based technique to dynamically tune the configuration parameters of the SAT solver during runtime. It is noteworthy that the generality of DCT significantly differentiates it from previous MAB-based approaches [2], [3], which predominantly address specific configurations. In contrast, DCT provides a comprehensive and adaptable framework that can be implemented with minimal engineering modifications to existing solver architectures.

III. Cold Restart

Restart is an essential technique employed in modern CDCL SAT solvers that contributes significantly to their performance efficacy. Despite extensive and in-depth research on restart strategies, most well-known CDCL solvers currently implement restart actions that only cancel the current variable assignment while maintaining all other state information. To our knowledge, only two recent studies [2], [4] have explored restart strategies that modify additional state information beyond variable assignments. [4] introduces a taxonomy distinguishing between "warm restart" (which only removes current variable assignment information) and "cold restart" (which additionally removes other state information). [2] introduces the term "reset" to describe restart strategies that remove variable activity scores in addition to current assignments. In our work, we adopt the terminology proposed in the former study. Both [4] and [2] have demonstrated experimental results that highlight the benefits of judiciously implementing cold restart strategies, particularly for satisfiable instances. Warm restart policies contribute significantly to assignment stack compaction [5] and improved quality of learned clauses [6]. Nevertheless, these approaches consistently redirect the search trajectory after a restart to regions adjacent to the pre-restart position in the search space. In contrast, cold restart methodologies substantially reduce the probability of continued entrapment within disadvantageous search regions [4], enabling more diversified exploration of potential solution spaces. [4] examined three distinct cold restart policies and their combinations: (1) FO policy: cold restart that forgets the branching order; (2) FP policy: cold restart that forgets the phases; and (3) FC policy: cold restart that forgets (part of the) learned clauses. To further validate and investigate the efficacy of cold restart strategies, we implemented all these policies in Kissat 4.0.2. We intend to select the most effective combination for participation in the main track of the SAT Competition 2025.

References

[1] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. Cadical, gimsatul, isasat and kissat entering the sat competition 2024. In *Proc. of SAT Competition (2024)*, pages 8–10, 2024.
[2] C Li, C Liu, J Chung, et al. A reinforcement learning based reset policy for cdcl sat solvers. *arXiv preprint arXiv:2404.03753*, 2024.
[3] Mohamed Sami Cherif, Djamal Habet, and Cyril Terrioux. Combining vsids and chb using restarts in sat. In *27th International Conference on Principles and Practice of Constraint Programming*, 2021.
[4] X Zhang, Z Chen, and S Cai. Revisiting restarts of cdcl: Should the search information be preserved? *arXiv preprint arXiv:2404.16387*, 2024.
[5] Youssef Hamadi, Saïd Jabbour, and Lakhdar Sais. Manysat: a parallel sat solver. *J. Satisf. Boolean Model. Comput.*, 6(4):245–262, 2009.

[6] Jia Hui Liang, Chanseok Oh, Minu Mathew, Ciza Thomas, Chunxiao Li, and Vijay Ganesh. Machine learning-based restart policy for cdcl sat solvers. In *SAT 2018*, pages 94–110, 2018.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Kissat_CoRephase_CoReward and Kissat_MAB_CoRephase solvers in SAT Competition 2025

Shuang Chen*
College of Computer Science and Technology
National University of Defense Technology
Changsha,China
chenshuang21a@nudt.edu.cn

Jianmin Zhang*
College of Computer Science and Technology
National University of Defense Technology
Changsha,China
jmzhang@nudt.edu.cn

Jinjin Liu*
College of Computer Science and Technology
National University of Defense Technology
Changsha,China
liujinjin @nudt.edu.cn

Yan Sun
College of Computer Science and Technology
National University of Defense Technology
Changsha,China
yansun@nudt.edu.cn

Tiejun Li
College of Computer Science and Technology
National University of Defense Technology
Changsha, China
tjli@nudt.edu.cn

***Abstract*—This document describes the SAT solver Kissat_CoRephase_CoReward and Kissat_MAB_CoRephase submitted to the 2025 SAT Competition.**

I. INTRODUCTION

The excellent SAT solver Kissat[1] designed six rephasing heuristics and scheduled them to form a rephasing strategy. The default rephasing policy of Kissat solver is $\{OI(BWOBWIBW\#BWF)^{\omega}\}$. Kissat_MAB_Conflict+[1] introduces a novel rephasing heuristic. By guiding the solver to detect more unsatisfiable cores and learn additional clauses, this approach combines the new heuristic with Kissat's original heuristics to generate a new rephasing strategy. Based on Kissat, Kissat-MAB-rephasing[1] employs an MAB-based rephasing strategy, which uses the UCB policy to select a rephasing heuristic from a set of available options. Based on these techniques, we have designed a novel rephasing strategy and implemented a solver called Kissat_MAB_CoRephase. Building upon Kissat_MAB_CoRephase, we implemented a solver named Kissat_CoRephase_CoReward, which adopts a dynamic reward function framework optimized across three targeted dimensions, achieving remarkable improvements in both solving efficiency and the number of solvable instances.

II. MAIN TECHNIQUE

A. Combining Rephasing heuristic

Based on Kissat, Kissat solver employs six rephasing strategy as the following equation:

$$A = \{\text{ Best , Inverted, Original, Walk, Flipped, Conflict }\} \tag{1}$$

The main objective of the first group of heuristics is to prevent the algorithm from getting stuck in local optima, while the second group emphasizes optimizing the search process to improve the efficiency of attaining the global optimum. By integrating these two clusters of heuristics, the CDCL algorithm can explore the solution space more efficiently, circumvent the risks related to local optima, and increase its robustness and adaptability when addressing complex issues. Additionally, combining the Best (B) heuristic and the Walk (W) heuristic may help the solvers find models of SAT instances faster. So, we integrate the BW heuristics and the heuristics from the first group. The candidate rephrasing strategies for the MAB are listed below.

$$A = \{BWO, BWI, BWC, BWF\} \tag{2}$$

B. Combining Reward Function

Three reward functions have been designed across the following dimensions:

Conflict Analysis Guidance

$$R_{\text{conflict}} = \frac{\log_2(\text{ decisions }_t)*\log_2(conflicts_t)}{\text{decidedVars }_t^2} \tag{3}$$

In equation 3, the variable t denotes the current run. In addition, $decisions_t$ and $\text{conflicts}_{\text{t}}$ denote the number of decisions and conflicts since the latest restart respectively. decidedVars_t^2 denote the number of variables that have been assigned a value so far. This function takes into account the number of decisions and conflicts during branching process.

Clause Quality Optimization:

$$R_{\text{clause}} = \frac{1}{1+\overline{LBD}} \tag{4}$$

Rewards clauses with average low Literals Blocks Distance ($\overline{LBD}$) and low redundancy, to enhance database pruning efficiency.

Remaining Variable Minimization:

$$R_{\text{variable}} = \frac{1}{1+reamain_vars} \tag{5}$$

*These authors contributed equally to this work.

Prioritizes assignments that reduce the number of unassigned variables, accelerating search space convergence.

The composite reward function integrating these three components is formulated as:

$$R_{\text{hybrid}} = \alpha R_{\text{conflict}} + \beta R_{\text{clause}} + \gamma R_{\text{variable}} \quad (6)$$

After some experimental optimization, the values of these hyperparameters are as follows: $\alpha = 0.5 * 300.0$, $\beta = 0.4 * 1.0$, $\gamma = 0.1 * 10.0$.

III. Using the Template

We have implemented this new rephasing heuristic in Kissat_MAB_CoRephase solver. Its source code is available at https://github.com/2317891476/Kissat_MAB_CoRephase.git. We have implemented both the new rephasing heuristic and the reward function in the Kissat_CoRephase_CoReward solver. Its source code is available at https://github.com/2317891476/Kissat_CoRephase_CoReward.git.

IV. ACKNOWLEDGEMENT

We would like to thank everyone who contributed to KISSAT, Kissat_MAB_Conflict+ and Kissat-MAB-rephasing for their efforts and for making their tools publicly available. This work was supported by National Natural Science Foundation of China under grant No.62072464.

References

[1] A Biere, M Fleury. Chasing target phases. In Workshop on the Pragmatics of SAT. 2020.

[2] J. Zheng, M. Jin, K. He, Z. Chen, and J. Xue. New rephasing strategies and their combinations. SAT COMPETITION 2023, p. 27

[3] X Chen, W Guo, W Luo, et al. Kissat-MAB-rephasing and Kissat relaxed [J]. SAT COMPETITION 2022: 35.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

A Self-Optimizing Framework for SAT Solvers via Population Evolution and Large Language Model Collaboration

1st Hang Ding
School of Computer Science
Hubei University of Technology
Wuhan, China
dinghang@hbut.edu.cn

2nd Mao Luo
School of Computer Science
Hubei University of Technology
Wuhan, China
luomao@hbut.edu.cn

3rd Chu-Min Li
MIS, University of Picardie Jules Verne
Amiens, France
chu-min.li@u-picardie.fr

4th Shunwei Li
School of Computer Science
Hubei University of Technology
Wuhan, China
lishunwei@hbut.edu.cn

5th Runyao Chen
School of Computer Science
Hubei University of Technology
Wuhan, China
chenrunyao@hbut.edu.cn

6th Caiquan Xiong
School of Computer Science
Hubei University of Technology
Wuhan, China
xiongcq@hbut.edu.cn

7th Xinyun Wu
School of Computer Science
Hubei University of Technology
Wuhan, China
wuxinyun@hbut.edu.cn

***Abstract*—This paper proposes the AE framework, an automated optimization approach that integrates collective learning and genetic evolutionary principles. Unlike conventional practices relying on single models or manual tuning, AE autonomously enhances solver performance through large-model interaction, eliminating the need for extensive human annotation or complex training processes.**

***Index Terms*—Automated optimization, Collective learning, Genetic evolution, Large-model interaction.**

I. Introduction

In the operational mechanism of Conflict-Driven Clause Learning (CDCL) SAT solvers, the selection of heuristic strategies directly determines system performance [1]. However, traditional heuristic methods inherently suffer from limited domain adaptability, making it challenging to construct universal solutions. While mainstream solvers (e.g., MiniSat and Kissat) employ hybrid heuristic strategies, their selection mechanisms often rely on empirical rules [2], requiring expert knowledge and extensive time for iterative tuning. To address this issue, this paper proposes the AE framework.

A. Hybrid of Experts (MoE) framework

AE employs a Hybrid of Experts (MoE) framework, specifically integrating the DeepSeek-R1 model for strategy analysis, the Claude3.7 model for code implementation, and the ChatGPT-4.5 model for strategy generation.

B. Collaborative Learning

The automated improvement of heuristic strategies in the AE framework follows this learning workflow:

Step I - Team Leader Generation: At each iteration, randomly generate n distinct heuristic improvement strategies as team leaders from promising candidate functions.

Step II - Team Member Generation: For each team leader's heuristic strategy, create m variant strategies as intra-team members through controlled mutation of the original strategy. These intra-team members variants maintain strategic similarity while introducing controlled differentiation, followed by individual performance evaluation for each variant.

Step III - Leader Re-selection: Within each team, select the optimal-performing team member strategy as the new team leader for the current iteration cycle.

Step IV - Iterative Update: Repeat Step II until reaching prespecified termination criteria (maximum iteration count).

C. Genetic Evolution

Building upon this foundation, the AE framework incorporates an independent genetic evolution process for enhanced strategy optimization:

Step I - Strategy Analysis: Employ large language models (LLMs) to conduct causal analysis of team leader strategies obtained from collaborative learning, identifying precise mechanisms underlying their performance improvements or degradations.

Step II - Hybrid Evolution: Compared to the random crossover combinations in genetic algorithms, this study fully leverages the strategic analysis from the first step. Based on a detailed examination of how the model influences outcomes, it employs the large language model to identify the most

promising strategy combinations for experimentation, followed by evaluation and strategic analysis.

Step III - Evaluation and Pruning: Retain the top-performing strategy. For the remaining candidates, employ LLMs to assess the innovation level ($c_{innovation}$) and structural complexity ($c_{complexity}$) Calculate composite scores:

$$\text{Score} = \left(\frac{T_{\text{base}}}{T_{\text{strategy}}}\right) + \alpha(c_{\text{innovation}} + c_{\text{complexity}}) + \beta r \quad (1)$$

(α, β are weighting coefficients; r = random perturbation)

Eliminate the lower 50

Step IV - Iterative Refinement: Repeat Step II until convergence to a single optimal strategy.

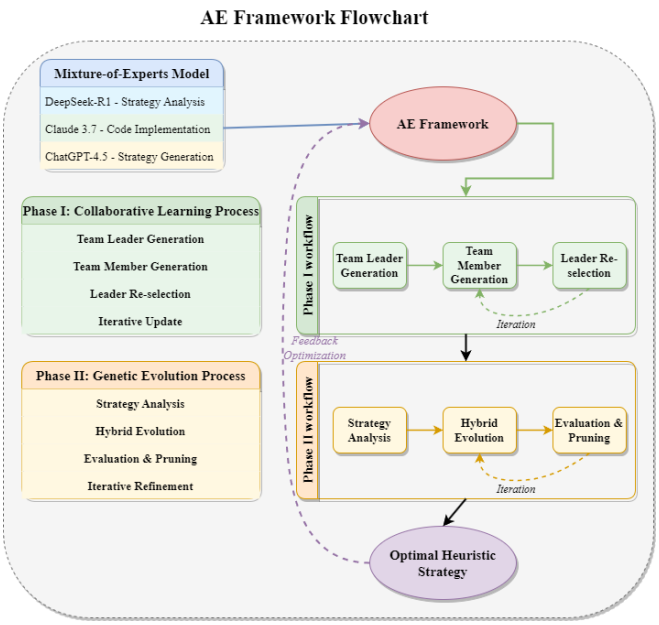


Fig. 1. AE Framework Flowchart.

II. Main work

Utilizing the aforementioned framework, we enhanced both the latest version of Kissat (version 4.0.2) and its modified variant incorporating the MAB strategy, thereby developing three improved SAT solving frameworks: AE_kissat2025_bump, AE_kissat2025_rescale, and AE_kissat2025_MAB.

A. AE_kissat2025_bump

The AE_kissat2025_bump framework implements modifications to the kissat_bump_score_increment() function. Through multiple rounds of iterative framework optimization, we identified an effective enhancement approach that introduces a dynamic hybrid piecewise strategy coupled with a compensation factor mechanism. This method operates by calculating the ratio of the current score increment to the predefined maximum value, thereby partitioning the decay coefficient (*dynamic_decay*) into two dynamically adjusted modes:

- For *ratio* < 0.5, a linear piecewise strategy is employed to enhance dynamic adaptability through *base_decay* × (0.5 + *ratio*).
- When *ratio* ≥ 0.5, the hyperbolic tangent (tanh) function's nonlinear characteristics are utilized to smoothly regulate the decay coefficient, effectively suppressing parameter fluctuations in high-ratio regions and mitigating excessive adjustments caused by extreme ratio values.

Furthermore, we construct a logarithmic compensation factor $\log(2.0 - dynamic_decay)$ and a linear compensation term $1.0 + 0.15 \times dynamic_decay$, combining them into a composite compensation term that optimizes the score increment growth rate across different phases. This innovation transforms the score increment factor (*factor*) computation from the legacy fixed-decay pattern to a dynamic self-adaptive mode, which simultaneously reduces computational overhead from frequent score rescaling operations and enhances algorithmic stability through smooth transition mechanisms. Experimental results demonstrate that this optimization strategy achieves approximately 5% improvement in PAR2 scores for 2024 benchmark instances.

B. AE_kissat2025_rescale

The AE_kissat2025_rescale framework implements modifications to the kissat_rescale_scores() .Through multiple rounds of iterative framework optimization, we identified an effective enhancement approach that introduces three distinct score scaling strategies coupled with a pseudo-random selection mechanism for dynamic switching, thereby optimizing the score adjustment process.Specifically, a pseudo-random number generator based on the current increment (`scinc`) determines the strategy selection:

$$\text{choice} = (\texttt{solver->scinc} \times 1000) \bmod 3$$

The modulus result (0–2) triggers the following scaling methods:

1. Maximum Reciprocal Method (`mod` = 0):

$$\text{factor} = \frac{1.0}{\max(\text{max_score}, \text{scinc})}$$

Preserves the original normalization logic.

2. Geometric Mean Reciprocal Method (`mod` = 1):

$$\text{factor} = \frac{1.0}{\sqrt{\text{max_score} \times \text{scinc}}}$$

Balances the parameter scales multiplicatively.

3. Logarithmic Mean Reciprocal Method (`mod` = 2):

$$\text{factor} = \frac{1.0}{\frac{\log(\text{max_score}+1)+\log(\text{scinc}+1)}{2}}$$

Suppresses extreme values through nonlinear averaging.

This hybrid strategy dynamically adapts to varying score distributions while maintaining computational efficiency.

C. AE_kissat2025_MAB

- The AE_kissat2025_MAB framework primarily modifies the restart_mab() function. Through multiple rounds of iterative framework optimization, an effective enhancement approach has been identified, which introduces a dynamic momentum-regulated adaptive exploration mechanism and constructs a gain-tracking feedback system. Specifically:
  1) Gain Sliding Window: A sliding window of length 10 (*recent_gains*) is introduced to track recent heuristic performance. By comparing the immediate gain of the current heuristic method (current_gain = $\frac{\text{reward}}{\text{select}}$) with the sliding window average (avg_gain), a dynamic momentum coefficient (momentum) is generated.
     - When current_gain > avg_gain, the momentum increases multiplicatively by $1.1\times$momentum, reinforcing the exploitation tendency of the current strategy.
     - When current_gain $\leq$ avg_gain, the momentum decays multiplicatively by $0.9 \times$ momentum, enhancing exploration capabilities.
  2) Adaptive Exploration Coefficient: The original fixed exploration coefficient (mabc) is replaced with a dynamic parameter:

     $$\text{adaptive_c} = \frac{\text{mabc}}{\text{momentum} \times (\text{stable_restarts} + 1)}$$

     This modification enables the exploration term in the UCB (Upper Confidence Bound) formula to adaptively adjust based on momentum and restart counts.
     - Stability-driven decay: As stable_restarts increases (indicating stable search phases), adaptive_c decreases to suppress excessive exploration.
     - Momentum compensation: When momentum decreases (indicating local optima), adaptive_c inversely compensates to reactivate diversified search.
- Experimental results demonstrate that this optimization strategy achieves approximately 3% improvement in PAR2 scores for 2024 benchmark instances.

References

[1] M. S. Cherif, D. Habet, and C. Terrioux, "Combining VSIDS and CHB Using Restarts in SAT," in *Proc. 27th Int. Conf. Principles and Practice of Constraint Programming*, 2021.

[2] Y. Sun, F. Ye, X. Zhang, S. Huang, B. Zhang, K. Wei, and S. Cai, "AutoSAT: Automatically Optimize SAT Solvers via Large Language Models," arXiv preprint arXiv:2402.10705, 2024.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Kissat_VSA in the SAT Competition 2025

Shuolin Li*, Jialu Zhang†,
*Aix Marseille Univ, Université de Toulon, CNRS, LIS, Marseille, France
shuolin.li@lis-lab.fr
†Université de Picardie Jules Verne, Amiens, France
jialu.zhang@u-picardie.fr

***Abstract*—This document is an introduction to the solvers submitted for the SAT 2025 Competition.**

I. Introduction

We introduce Kissat_VSA, a solver submitted to this year's SAT Competition, which is based on last year's winning solver Kissat-sc2024 [1]. The acronym VSA stands for clauses Vivification Scheduled by Activity, a technique where clause vivification is guided by the branching heuristic scores of variables. Specifically, we sort clauses by the minimum heuristic score (or activity) among their variables, in descending order, and prioritize vivification on clauses with the highest minimum activity. Note that the ordering of variables within each clause remains unchanged.

In Kissat, VSA is activated only when the solver is in stable mode, meaning the VSIDS heuristic is in use rather than VMTF. Outside of this mode, the original vivification method from Kissat is used instead.

References

[1] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "Cadical, gimsatul, isasat and kissat entering the sat competition 2024," *Proc. of SAT Competition*, pp. 8–10, 2024.

Acknowledgment

This work is supported by AI CHAIR reference ANR-19-CHIA-0013-01 (MASSAL'IA) funded by the French Agence Nationale de la Recherche and the electricity distribution network manager Enedis (project partner), and partially supported by Archimedes Institute, Aix-Marseille University. We want to thank the Université de Picardie Jules Verne for providing the Matrics Platform.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Kissat_Scavel_RS and Kissat_Scavel2025

1st Zhihui Li, 2nd Guanfeng Wu, 3rd Li Chen,4rd Shuwei Chen, 5th Yang Xu
School of Mathematics,
National-Local Joint Engineering Laboratory of System Credibility,
Southwest Jiaotong University,
Chengdu, China
lizhihui@swjtu.edu.cn, wgf1024@swjtu.edu.cn, cli1004@my.swjtu.edu.cn, swchen@swjtu.edu.cn, xuyang@swjtu.edu.cn

***Abstract*—This document describes Kissat_Scavel_RS and Kissat_Scavel2025 at the SAT Competition 2025.**

I. Introduction

The base solvers we used to implement our techniques are Kissat-rel-4.0.1 and Kissat-sc2024 , obtained from the SAT Competition 2024 [1]. Based on these very competitive solvers, we made some minor changes.

II. Inverse Restart Enhancement technique

Modern SAT solvers exhibit well-established core components predominantly based on forward deduction mechanisms. We investigate the reverse utilization of existing reasoning sequences and their integration with solver restart mechanisms. This work proposes a phased Inverse Boolean Constraint Propagation (BCP) Restart Enhancement technique (IPRE), which leverages staged reasoning artifacts – specifically propagation implication graphs – to amplify leaf node activation values and enable inverse deduction of decision branches. Implemented in the state-of-the-art solver kissat-sc2024-d776b9e1, our enhanced system is rebranded as kissat_scavel2025.

III. Reward Score

Many state-of-the-art SAT solvers adopt the VSIDS heuristic for variable selection during decision-making. However, this approach has certain drawbacks. Primarily, VSIDS evaluates variables solely based on their recent involvement in conflicts, overlooking their inherent structural significance within the problem. Additionally, since it prioritizes variables that have been frequently involved in recent conflicts, the solver may become trapped in a local optimum.

To overcome these limitations and enhance the effectiveness of variable selection, we aim to identify structurally significant variables earlier in the search process and boost their activity scores more substantially when conflicts arise. With this motivation, we propose augmenting the original activity score of a variable by incorporating a reward term that reflects its importance. The formula for the proposed variable score adjustment is presented below:

$$S_{new} = S_{old} + S_{reward} \tag{1}$$

This work is also supported by the National Natural Science Foundation of China (Grant No: 62106206).

S_{new} refers to the variable activity score after adding the bonus score, while S_{old} refers the score before the addition.S_{reward} represents the reward score assigned to the variable.

$$S_{reward} = \alpha \times W_{lbd} \times freq \times sumConflicts \tag{2}$$

- W_{lbd} is the inverse of the LBD value, as previously defined.
- $freq$ represents the count of occurrences of the variable in conflict clauses.
- $sumConflicts$ denotes the current total number of conflicts.
- α is a tuning parameter.

We named this version Kissat_Scavel_RS.

Acknowledgment

The author would like to thank the developers of Kissat, and all the authors who contributed to the modifications that have been integrated. Their solvers are the foundation of our learning and improvement. This work is supported by the National Natural Science Foundation of China (Grant No:62106206).

References

[1] SC 2024, https://satcompetition.github.io/2024/

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

KISSAT-PRED and KISSAT-PRED-AGGRESSIVE in SAT Competition 2025

Chuan Luo, Zhenwei Yang, Shaoke Cui, Chunming Hu
School of Software
Beihang University
Beijing, China
{chuanluo, zwyang, cuisk, hucm}@buaa.edu.cn

I. INTRODUCTION

Studies have shown that features extracted from the graphs induced by the instances preserve certain satisfiability information [1], [2]. The strategies that solvers use to find a SAT assignment are different from those used to prove UNSAT [3]. Predictions based on the features can be used to adjust solving strategies adaptively, thereby improve the solver performance.

KISSAT-PRED and KISSAT-PRED-AGGRESSIVE are built on the latest KISSAT solver [4]. KISSAT-PRED predicts the satisfiability of the preprocessed instance and applies corresponding searching strategies to accelerate the solving process. Unlike KISSAT-PRED, KISSAT-PRED-AGGRESSIVE uses more aggressive strategies.

II. KISSAT-PRED

In KISSAT-PRED, features used for prediction are the normalized statistical measurements of node degrees or edge weights in the graphs. The graphs including Clause Graph, Variable Graph and Clause-Variable Graphs are generated after the preprocessing phase. Feature extraction runs in linear time.

After analyzing the importance of each feature, we identified the most important features for prediction. The rest of the features are ignored, as they may introduce randomness into the predictions. Decision trees are trained on publicly available instances to predict the satisfiability and difficulty of the instance. The trained decision trees are then quantized and embedded into the solver.

KISSAT-PRED applies corresponding strategies to improve the solving performance for instances predicted as difficult SAT or UNSAT, and uses default strategies for UNKNOWN-predicted instances. Compared to the aggressive version, KISSAT-PRED performs more conservatively, only adjusting restart intervals and mode-switching periods.

III. KISSAT-PRED-AGGRESSIVE

KISSAT-PRED-AGGRESSIVE shares the same prediction process as KISSAT-PRED. The difference between KISSAT-PRED-AGGRESSIVE and KISSAT-PRED is that KISSAT-PRED-AGGRESSIVE uses more advanced and aggressive strategies than KISSAT-PRED during solving. For example, KISSAT-PRED-AGGRESSIVE allocates more resources to clausal congruence closure [5] for UNSAT-predicted instances in order to reduce the size of the instance.

ACKNOWLEDGMENT

The authors would like to thank the developers of the KISSAT solver, and all the authors who contributed to the modifications that have been integrated. Their solver is the foundation of our improvement.

REFERENCES

[1] Enrique Matos Alfonso and Norbert Manthey, "New CNF Features and Formula Classification", POS@SAT, pp. 57–74, 2014.
[2] Marco Dalla, Benjamin Provan-Bessell, Andrea Visentin, and Barry O'Sullivan, "SAT Feature Analysis for Machine Learning Classification Tasks", in Proceedings of the International Symposium on Combinatorial Search, 2023, vol. 16, pp. 138–142.
[3] Oh Chanseok, "Between SAT and UNSAT: the fundamental difference in CDCL SAT", International Conference on Theory and Applications of Satisfiability Testing. Cham: Springer International Publishing, pp. 307–323, 2015.
[4] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks and Florian Pollitt, "CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2024", Proc. SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions, vol. B-2024-1, pp. 8–10, 2024.
[5] Armin Biere, Katalin Fazekas, Mathias Fleury and Nils Froleyks, "Clausal Congruence Closure", in Proceedings of SAT 2024, pp. 6:1–6:25, 2024.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

PRS: A parallel framework for SAT

Yuhang Qian[1, 2], Zhihan Chen[1, 2], Xindi Zhang[1, 2], Shaowei Cai[1, 2, *]
[1] *Key Laboratory of System Software (Chinese Academy of Sciences) and State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences*
[2] *School of Computer Science and Technology, University of Chinese Academy of Sciences*
{qianyh, chenzh, zhangxd, caisw}@ios.ac.cn

I. Introduction

PRS [4] is a lightweight parallel framework PRS, which can be seen as an improved version of ParKissat-RS [6]. PRS enters both the parallel track and cloud track, which is developed mainly based on Kissat-Inc [3], and uses many pre-processing techniques of Kissat-Pre [3].

In last year's Painless solver [5], SBVA was proven to be an effective strategy; therefore, we have also integrated this technique into our solver.

In this year's competition, we plan to submit two versions: PRS-SC25 without SBVA, and PRS-SC25-SBVA with the SBVA technique.

II. PRS

PRS is a simple, efficient, and generic parallel SAT framework. PRS is a parallel portfolio framework that supports pre-processing, clause sharing, and many other popular parallel techniques. The diversity for each thread mainly comes from randomly shuffling (RS) [6] the initial branching order. On the base of PRS framework, we developed the following two versions for the parallel track.

A. PRS-SC25

PRS-SC25 is built upon Kissat-Inc, whose clause sharing strategy is the same as HordeSat [1]. PRS uses many pre-processing techniques, which can be found in [6]. RS is used in default to introduce diversity for each thread. In detail, PRS-sc25 set up a circular queue that stores all the variables in the order of their internal indices in Kissat-Inc, and the circular queue is equally divided into n blocks, $B_1, ..., B_n$. For the i-th thread, the initial variable branching order begins from B_i and ends with B_{i-1}. Note that n is the number of threads, the next block of B_n is B_1, and B_{i-1} is B_n when $i = 0$.

While one thread runs PRS preprocessing, the other 31 threads run yalsat for local search using different random seeds. The best phase from each thread is used as the initial phase for the kissat solver, except for thread 0, which follows kissat's default phase strategy.

Specifically, we introduced the SWEEP feature to PRS-SC25. This feature originally appeared in kissat-sc24 [2], and we have ported it to Kissat-Inc.

This work was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDA0320000 and XDA0320300.

B. PRS-SC25-SBVA

Similar to the approach used in last years Painless solver [5], PRS-SC25-SBVA builds upon the 20-thread PRS by integrating 2 threads of YalSAT and 10 threads for solving SBVA-reduced subproblems, resulting in a total of 32 parallel threads.

Building on PRS-SC25, we assign a subset of threads (11 threads) to first perform SBVA simplification on the formulas before solving, while the remaining threads (21 threads) start solving immediately after an initial PRS-based simplification. Both types of threads have the SWEEP feature enabled.

References

[1] T. Balyo, P. Sanders, and C. Sinz. Hordesat: A massively parallel portfolio sat solver. In *SAT 2015*, pages 156–172, 2015.
[2] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt. Cadical, gimsatul, isasat and kissat entering the sat competition 2024. *Proc. of SAT Competition*, pages 8–10, 2024.
[3] Z. Chen, X. Zhang, S. Cai, and P. Lu. Cdcl solvers with improved local search cooperation and pre-processing. *SAT COMPETITION 2022*, page 37.
[4] Z. Chen, X. Zhang, Y. Qian, and S. Cai. Prs: A new parallel/distributed framework for sat. *SAT COMPETITION 2023*, page 39.
[5] M. Saoudi, S. Baarir, J. Sopena, T. Lejemble, and S. Saouli. Painless in sat competition 2024 with pl-prs-bva-kissat and pl-prs-mpi. *SAT COMPETITION 2024*, page 24.
[6] X. Zhang, Z. Chen, and S. Cai. Parkissat: Random shuffle based and pre-processing extended parallel solvers with clause sharing. *SAT COMPETITION 2022*, page 51.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Reencoding Unique Literal Clauses in CaDiCaL

Joseph E. Reeves Aeacus Sheng

***Abstract*—We present a lightweight reencoding technique implemented as a preprocessing module inside of the SAT solver CaDiCaL. The technique first detects implicit or explicit exactly-one constraints, which we name unique literal clauses (ULCs) and exclusive literal clauses (XLCs). Then, the literals within the ULCs and XLCs are sorted based on an algorithm that performs alignment via binary clauses. Finally, the ULCs and XLCs are reencoded as the corresponding sequential counter exactly-one encoding or order encoding. The introduction of auxiliary variables from this reencoding will enable the solver to reason about groups of objects; and crucially, these reasoning capabilities are enhanced by an alignment between encodings.**

I. Overview

We present a preprocessing technique that detects unique literal clauses within a formula then reencodes them as the sequential counter exactly-one encoding or the order encoding.

Definition 1. *A clause C is a* unique literal clause (ULC) *with respect to a formula Γ if none of the literals $\ell \in C$ occurs in $\Gamma \setminus \{C\}$.*

Given a formula Γ, the set of ULCs can be computed in linear time in the size of Γ by first computing which literals occur only once in Γ and then determining which clauses only contain such literals.

A ULC can be interpreted as an implicit exactly-one constraint. To see this, consider a graph-coloring problem:

Example 1. *With graph $G = (V, E)$ and k colors, let v_i denote whether vertex v has color i. The clauses of the graph-coloring problem are:*

- *for each $v \in V$: $\bigvee^{i \in k} v_i$*
- *for each $(u, v) \in E$: $\bigwedge^{i \in k} (\overline{u}_i \vee \overline{v}_i)$*

The first set of clauses stating that every vertex has at least one color are ULCs. Since the positive literals do not appear elsewhere in the formula, we can derive the AMO constraints stating that every vertex has at most one color. Combining the AMO with the existing clauses gives exactly one constraints, stating that each vertex has exactly one color.

II. Encodings

In our approach, we choose to reencode the exactly one constraint using the sequential counter. The **sequential counter encoding** by Sinz [3] for ExactlyOne constraints facilitates a compact representation using auxiliary variables s_i expressing that exactly one literal out of of the first i literals is true. The encoding uses the following definitions: $s_i \leftrightarrow (s_{i-1} \vee \ell_i)$ for $1 < i < k$ and $s_1 \leftrightarrow \ell_1$. Additionally, to enforce that at most one can be true, the clauses $\overline{s}_{i-1} \vee \overline{\ell}_i$ are included for $1 < i < k$. Finally, at least one of them must be true, which is encoded as follows: $\overline{s}_{k-1} \rightarrow \ell_k$.

Furthermore, if we chose to eliminate the literals v_i from the sequential counter encoding using variable elimination (DP resolution), we would end up with the order encoding. The **order encoding** [4] uses order variables $o_{\leq i}$ with $1 < i < k$ to express if one of $x_1, \ldots, x_i$ is `true`. To state that ℓ_i is `true` using only order variables, $o_{\leq i}$ must be `true`, while $o_{\leq i-1}$ must be `false`. The case for x_1 is special: ℓ_1 is `true` if and only if $o_{\leq 1}$ is `true`. When encoding ExactlyOne, the case ℓ_k is also special: ℓ_k is `true` if and only if $o_{\leq k+1}$ is `false`. The order encoding uses the following clauses $\overline{o}_{\leq i} \vee o_{\leq i+1}$ stating that if one of the first i literals is `true`, then one of the first $i+1$ is `true`.

The order encoding eliminates the x_i variables by replacing all their occurrences with $o_{\leq i}$ variables. More specifically,

- all literals ℓ_1, $\overline{\ell}_1$, ℓ_k, and $\overline{\ell}_k$ are replaced by $o_{\leq 1}$, $\overline{o}_{\leq 1}$, $\overline{o}_{\leq k-1}$, and $o_{\leq k-1}$, respectively.
- all literals $\overline{\ell}_i$ with $1 < i < k$ are replaced by $o_{\leq i-1} \vee \overline{o}_{\leq i}$.
- all clauses with literals ℓ_i with $1 < i < k$ are duplicated with one copy using $\overline{o}_{\leq i-1}$ and the other copy using $o_{\leq i}$ instead of ℓ_i.

Example 2. *For the order encoding of a graph-coloring problem with graph $G = (V, E)$ and k colors, let $v_{\leq i}$ denote whether vertex v has color at most i. The clauses are:*

- *For each $v \in V$: $(\overline{v}_{\leq 1} \vee v_{\leq 2}) \wedge (\overline{v}_{\leq 2} \vee v_{\leq 3}) \wedge \cdots \wedge (\overline{v}_{\leq k-2} \vee v_{\leq k-1})$*
- *For each $(u, v) \in E$: $(\overline{u}_{\leq 1} \vee \overline{v}_{\leq 1}) \wedge (u_{\leq 1} \vee \overline{u}_{\leq 2} \vee v_{\leq 1} \vee \overline{v}_{\leq 2}) \wedge \cdots \wedge (u_{\leq k-1} \vee v_{\leq k-1})$*

Note that for larger k, the clauses of length four will outnumber the binary clauses.

III. Clashes and Generalizations

Prior to reencoding, we remove any clashing ULCs. Two unique literal clauses C and D are said to *clash* (on a literal ℓ) if there exist $\ell \in C$ and $\overline{\ell} \in D$. We also say that they clash on the variable v underlying ℓ. Note that resolving two clashing ULCs either results in a tautology (if there are multiple clashing pairs) or a new ULC. So, if a formula has a pair of clashing ULCs, we can either just eliminate the pair (if the resolvent is a tautology) or replace the pair with a new one while eliminating a variable.

We can consider a generalization of ULCs.

Definition 2. *Given a formula Γ and a clause C. Let $U(C, \Gamma)$ be the set of literals in C that do not occur in $\Gamma \setminus \{C\}$ and $X(C, \Gamma) = C \setminus U(C, \Gamma)$. A clause C is an* exclusive literal clause (XLC) *with respect to Γ if $(\overline{\ell} \vee \overline{\ell'}) \in \Gamma$ for all $\ell, \ell' \in X(C, \Gamma)$.*

The closest related work is by Manthey and Steinke [2], who also propose transforming the direct encoding into the sequential counter encoding. They detect explicit direct encodings of

EXACTLYONE constraints rather than ULCs to guarantee a reduction in formula size. So, a ULC is an XLC in which all literals are unique, while Manthey and Steinke detect XLCs with no unique literals. We do not remove clashes in XLCs that are not ULC since variable elimination could blow up the size of the formula; in certain cases we must choose which of two clashing XLCs to reencode. In the following sections, we discuss alignment of ULCs, but this can adapted to handle XLCs as well.

IV. ALIGNMENT OF ULCS

The effectiveness of our reencoding heavily depends on the order of literals in ULCs. Since ULCs are clauses, the literals can be permuted in any way to optimize performance. Since the order of literals matters, we consider ULCs as a sequence of literals in this section. Moreover, we assume that a formula is clash-free by applying any necessary resolutions.

Definition 3. *Let Γ be a formula whose exclusive literal clauses are $\{C_1, \ldots, C_m\}$. We say that Γ is aligned if for every pair of binary clauses $(\overline{a} \vee \overline{b}), (\overline{c} \vee \overline{d}) \in \Gamma$ with $a, c \in C_i$ and $b, d \in C_j$, it holds that literal a occurs before c in C_i if and only if b occurs before d in C_j.*

Definition 4. *A formula is* independent *if it is trivially aligned, that is, it does not contain any binary clauses between its ULCs. Suppose Γ is not independent. Then it is* alignable *if there exists a permutation of the literals in its XLCs such that it becomes aligned, and* unalignable *otherwise.*

To perform alignment, we first construct a graph G where nodes are literals and the graph contains an undirected edge between two literals ℓ and ℓ' iff $(\overline{\ell}' \vee \overline{\ell}) \in \Gamma$ and both ℓ and ℓ' occur in distinct ULCs. Next, we sort the ULCs from largest to smallest, sort the literals within each individual ULC by the natural ordering (ordering based on variable IDs), and initialize the alignment of all literals to 0. Then, for each $C \in XLCs$ and for each $\ell \in C$: if ℓ is not aligned, ℓ is assigned $alignment + 1$ and all unaligned literals in the connected component of G containing ℓ are aligned with $alignment + 1$. We increment $alignment$, then continue to the next literal. After aligning all literals in C, the clause is sorted based on the alignment values. If two literals from an ULC occur in the same connected component the problem is unalignable. In this case, the literals are sorted lexicographically by alignment and then by natural ordering. If no alignment occurs and the formula is independent (i.e. G is empty), then each clause retains its natural literal ordering.

Whether a formula is alignable is computable in linear time in the size of the formula. Since we can efficiently determine whether a formula is alignable, we can run this check first and only reencode alignable formulas.

V. PROOF PRODUCTION

Proof production uses a combination of reverse unit propagation (RUP) and resolution asymmetric tautology (RAT) steps. We make use of the extension stack inside of CaDiCaL to ensure solutions are reconstructed correctly. The general steps are as follows:

1) If the XLC contains unique literals, add remaining clauses of the direct encoding of the AMO on literals in the XLC
2) Add the exactly one sequential counter encoding, along with additional clauses in the proof necessary for the derivation
3) Delete the direct encoding of the AMO, and delete the XLC
4) Optionally, perform variable elimination to create the order encoding

VI. IMPLEMENTATION

We implemented the ULC and XLC reencoding technique as a preprocessing module within CaDiCaL [1], found in the file ORDERENCODE.CPP. Our preprocessor is called immediately before entering the CDCL loop, ensuring lucky search and other techniques can solve trivial problems prior to any attempted reencoding. We provide options for selecting which ULCs to reencode: alignable, independent, and unalignable, as well as whether or not to reencode XLCs. Resolution is performed on clashing ULCs before reencoding. Additionally, we provide the option to reencode as a sequential counter, and additionally to reencode as the order encoding for ULCs. All options support DRAT proof logging.

We submit the following four configurations:

- CaDiCaL-Alignable-ULC : Alignable ULC detection, alignment, and reencoding with the sequential counter
- CaDiCaL-Alignable-Independent-ULC : Alignable and independent ULC detection, alignment, and reencoding with the sequential counter
- CaDiCaL-Alignable-ULC-order : Alignable ULC detection, alignment, and reencoding with the order encoding
- CaDiCaL-XLC : All XLC detection, alignment, and reencoding with the sequential counter

REFERENCES

[1] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In Arie Gurfinkel and Vijay Ganesh, editors, *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024.

[2] Norbert Manthey and Peter Steinke. Quadratic direct encoding vs. linear order encoding a one-out-of-n transformation on cnf. Technical Report KRR Report 11-03, Technische Universität Dresden, 2011.

[3] Carsten Sinz. Towards an optimal cnf encoding of boolean cardinality constraints. In Peter van Beek, editor, *Principles and Practice of Constraint Programming - CP 2005*, pages 827–831, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[4] Naoyuki Tamura, Akiko Taga, Satoshi Kitagawa, and Mutsunori Banbara. Compiling finite linear CSP into SAT. *Constraints An Int. J.*, 14(2):254–272, 2009.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Painless in SAT Competition 2025 with PL-PRS-KISSAT and PL-PRS-GASPI-KISSAT

Mazigh Saoudi*, Souheib Baarir‡, Julien Sopena†,
Thibault Lejemble*, Sabrine Saouli†,
* EPITA, LRE, Kremlin-Bicêtre, France
†Sorbonne Université, LIP6, CNRS, UMR 7606, Paris, France
‡Université Paris Nanterre, France

***Abstract*—This paper describes the solvers PL-PRS-KISSAT and PL-PRS-GASPI-KISSAT submitted to the parallel track of the SAT Competition 2025.**

I. Introduction

Both submitted solvers are concurrent clause sharing portfolio solvers [1], [2] built using the Painless framework [3]. The sequential engines spawned in this solver are instances of Kissat [4]. Both solvers use the preprocessing techniques implementation of PRS [5] before launching the different solvers. PL-PRS-GASPI-KISSAT uses a genetic algorithm (GASPI [6]) to initialize half of the sequential Kissat's phases

II. Parallel Solvers

A. Setting-up Kissat

As it is necessary to introduce significant diversification in the portfolio to ensure that each instance of the sequential algorithm has a different execution, Kissat uses the same family configuration as our last years' submission [7] with each Kissat enables a certain subset of the 5 available preprocessing options. However, in order to have a sound clause sharing, the BVA inprocessing is disabled.

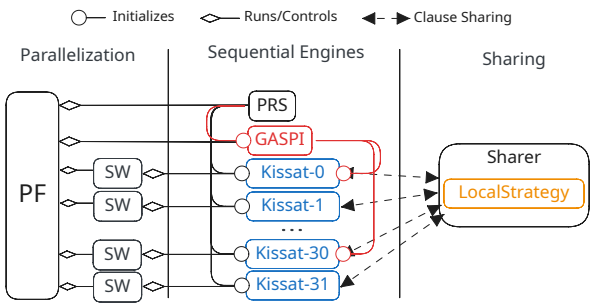


Fig. 1. Architecture of PL-PRS-GASPI-KISSAT

B. GASPI

GASPI is a genetic algorithm to find an optimal or near-optimal complete assignment that satisfies all or most of the clauses. If no model is found after a predefined number of generations, it assigns the preferred polarity of the variables based on the best values found (the values that satisfy most of the clauses). In PL-PRS-GASPI-KISSAT, one instance of GASPI is launched, and the final generation's population is used to initialize half of the Kissat sequential solvers.

C. Sharing strategy

In both solvers, the sharing strategy is inspired by the one used in [8] and [9]. A producer selects the clauses to be exported following a dynamic LBD threshold. At the beginning of the solving process, the threshold is LBD = 2. If the **Local Strategy** finds that a specific solver is sharing too few or too many clauses, then it increase or decrease this threshold in consequence. Every 0.5 seconds, the **Local Strategy** selects 1500 literals from each producer and dispatch them to consumers.

References

[1] Y. Hamadi, S. Jabbour, and L. Sais, "Manysat: a parallel sat solver," *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 6, pp. 245–262, 2009.
[2] D. Schreiber and P. Sanders, "MallobSat: Scalable SAT solving by clause sharing," *Journal of Artificial Intelligence Research (JAIR)*, 2024. Presented at Pragmatics of SAT (PoS) 2024.
[3] L. Le Frioux, S. Baarir, J. Sopena, and F. Kordon, "Painless: a framework for parallel sat solving," in *Proceedings of the 20th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 233–250, Springer, 2017.
[4] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024," in *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions* (M. Heule, M. Iser, M. Järvisalo, and M. Suda, eds.), vol. B-2024-1 of *Department of Computer Science Report Series B*, pp. 8–10, University of Helsinki, 2024.
[5] Z. Chen, X. Zhang, Y. Qian, and S. Cai, "Prs: A new parallel/distributed framework for sat," in *SAT COMPETITION 2023 Proceedings*, pp. 39,40, 2023.
[6] S. Saouli, S. Baarir, and C. Dutheillet, "Tackling the polarity initialization problem in sat solving using a genetic algorithm," in *NASA Formal Methods* (N. Benz, D. Gopinath, and N. Shi, eds.), (Cham), pp. 21–36, Springer Nature Switzerland, 2024.
[7] M. Saoudi, S. Baarir, J. Sopena, T. Lejemble, and S. Saouli, "Painless in sat competition 2024 with pl-prs-bva-kissat and pl-prs-mpi," in *SAT COMPETITION 2024 Proceedings*, pp. 23,24, 2024.
[8] T. Balyo, P. Sanders, and C. Sinz, "Hordesat: A massively parallel portfolio sat solver," in *Proceedings of the 18th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 156–172, Springer, 2015.
[9] Y. Hamadi, S. Jabbour, and J. Sais, "Control-based clause sharing in parallel sat solving," in *Autonomous Search*, pp. 245–267, Springer, 2011.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

MallobSat naps in the SAT Competition 2025

Dominik Schreiber Niccolò Rigi-Luperti
Institute of Information Security and Dependability
Karlsruhe Institute of Technology
Karlsruhe, Germany
{dominik.schreiber , niccolo.rigi-luperti}@kit.edu

Armin Biere
Chair of Computer Architecture
University of Freiburg
Freiburg i.Br., Germany
biere@cs.uni-freiburg.de

***Abstract*—We present the 2025 version of the parallel and distributed SAT solver MALLOBSAT to participate in this year's parallel track of the SAT Competition.**

***Index Terms*—SAT solving, HPC, preprocessing**

I. INTRODUCTION

Since SAT Competition 2025 does not feature a massively parallel / distributed / cloud track, the distributed solving features of MallobSat [7] remain un(con)tested this time – hence one might say that our system takes a nap this year. Still, we prepared a lightweight submission of single-process MallobSat to the parallel track. We use the same general setup and process layout as MallobSat in last year's parallel track [4] and integrated the 2024 solver backend of Kissat [2].

II. NAPS

We introduce a new solving approach we call "naps" ("*not a portfolio solver!*"). The central features of this setup are:

- We run four YalSAT [1] local search solvers and $p-4 = 60$ *uniform* Kissat solver threads. Across all threads, all pre– and inprocessing features are switched off, as well as the solver-specific ("native") cycled diversification of earlier years. The only kind of diversification we employ for the Kissat threads is very light: setting different random seeds and sparsely setting some random variable phases. This setup clearly departs from a classical "solver portfolio" and, in our view, should rather be considered a uniform parallel CDCL procedure with clause sharing acting as search space pruning. The few additional local search threads serve as independent "needle-in-the-haystack" searchers for a satisfying assignment.
- We employ a singular a-priori sequential preprocessor (not unlike the BVA performed in PL-PRS-BVA-KISSAT [3]), which uses Kissat's full preprocessing arsenal, as well as some preprocessing in Lingeling (Cardinality Constraint reasoning and Gaussian elimination). This allows to at times immediately recognize (un)satisfiability or, otherwise, simplify the formula. While this preprocessing is running, we already run a solving task at all (other) threads on the original formula. When preprocessing is done, we evict this solving task and replace it with a solving task that operates on the preprocessed formula instead. Any model reported by the latter task is transformed back into a model for the original formula.
- We replace MallobSat's strategy of incrementing incoming clauses' LBD values [7] with the simpler strategy of resetting all LBD values of clauses to their maximum once they are produced. This functionally disables all roles of LBD values in our clause sharing – in recent experiments we did not observe any loss in performance this way, while it simplifies the sharing logic.

We describe and evaluate (a slightly earlier version of) this setup in depth in our SAT'25 publication [6]. Note that Kissat is the only CDCL implementation we employ – we only use Lingeling's preprocessing and local search (YalSAT). As a side effect, this reduces the active code, simplifying maintenance and streamlining future research (also and especially in the distributed setting). We do not submit a version of our real-time proof-checking setup [4, 5] since the preprocessing does not yet support proof production. As a next step, we intend to replace our current preprocessing with proof-producing preprocessing and integrate it in our trusted solving pipeline.

ACKNOWLEDGMENT

The authors gratefully acknowledge the Gauss Centre for Supercomputing e.V. (www.gauss-centre.eu) for funding this project by providing computing time on the GCS Supercomputer SuperMUC-NG at Leibniz Supercomputing Centre (www.lrz.de). Research reported in this work was supported by an Amazon Research Award (Fall 2023).

REFERENCES

[1] Armin Biere. "Yet another local search solver and Lingeling and friends entering the SAT Competition 2014". In: *SAT Competition*. 2014, p. 65.
[2] Armin Biere et al. "CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024". In: *SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions* (2024), pp. 8–10.
[3] Mazigh Saoudi et al. "PL-PRS-BVA-KISSAT in SAT Competition 2024". In: *Pragmatics of SAT*. 2024. URL: https://dl.lre.epita.fr/papers/saoudi.24.pos.pdf.
[4] Dominik Schreiber. "MallobSat and MallobSat-ImpCheck in the SAT Competition 2024". In: *SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions* (2024), pp. 22–23.
[5] Dominik Schreiber. "Trusted Scalable SAT Solving with on-the-fly LRAT Checking". In: *Theory and Applications of Satisfiability Testing (SAT)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. 2024, 25:1–25:19. DOI: 10.4230/LIPIcs.SAT.2024.25.
[6] Dominik Schreiber, Niccolò Rigi-Luperti, and Armin Biere. "Streamlining Distributed SAT Solver Design". In: *Theory and Applications of Satisfiability Testing (SAT)*. 2025, 23:1–23:23. DOI: 10.4230/LIPIcs.SAT.2025.23.
[7] Dominik Schreiber and Peter Sanders. "MallobSat: Scalable SAT Solving by Clause Sharing". In: *Journal of Artificial Intelligence Research* 80 (2024), pp. 1437–1495. DOI: 10.1613/jair.1.15827.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

dilemma: A SAT-solver using a dilemma calculus on And-Inverter-Graphs

Benjamin Takacs

***Abstract*—This system description document presents `dilemma`, a SAT-solver using a dilemma calculus on And-Inverter-Graphs written in rust.**

The SAT-solver is named `dilemma` because it implements a dilemma calculus.

I. Calculus

In a dilemma calculus [2] we derive consequences of the formulae in study, by using it as assumption and chaining `simple rules` and a special name-giving dilemma rule schema. `Simple rules` are non-branching rules, that derive new consequences, for example one such rule could be: if $c = a \wedge a$ then $c = a$. Depending on the representation of the formulae only having non-branching rules is not enough, but we want to keep the property, that we get a set of consequences at the end of our derivation.

That is where the dilemma rule schema comes in. We use a case distinction and do a derivation for all the cases and analyse at the end for common consequences, as these are independent of the added assumption of the case distinction (we just didn't find them without).

We try to derive some inconsistency, like $a = \neg a$ for some literal a, to show that the starting formulae had to be inconsistent. Even if not run to completion, we can derive consequences of the starting formulae, so doing a preprocessing step with the dilemma calculus is possible. For example only allowing two nested dilemma applications.

For `dilemma` we represent the starting formulae as And-Inverter-Graph and consequences as equivalence relations. Equivalence relations have the advantage, that they can be represented using $O(n)$ memory and not growing even when more equivalences get added.

Other formulae and consequences representations are possible, but can need changes to the simple rules [3, 1].

II. Proof search

Like Stålmarck's proof procedure, we use iterative depth first search, where depth is the number of nested dilemma rule applications. For debugging and later a proof format, `dilemma` emits each successful rule application that locally lead to changes.

We naively try to apply all simple rules to all subformulae in each dilemma step to ensure completeness.

III. Datastructures

An And-Inverter-Graph (AIG) represents a set of formulae of the form $c = a \wedge b$ where a and b are literals and c is a positive literal and there is at most one formulae for each literal in the position of c. So we can say a literal c gets defined by that one formulae. And we can call literals a for which no such defining formulae exists `input literals`. By densely numbering the literals with the input literals first, we can represent the graph, by storing the number for the last input literal and then storing only the a, b pairs that define each c.

We can translate a CNF to an AIG by using a generalised form of De Morgans law $\bigvee(l_1, \ldots, l_n) = \neg \bigwedge(\neg l_1, \ldots, \neg l_n)$ and translating the arbitrary inputs $\bigwedge$ into a tree of two inputs $\wedge$ and assigning a new literal for each of these.

We use a variant of union-find to represent equivalence relations. As we have equivalences between literals $l_1 = l_2$ implies $\neg l_1 = \neg l_2$, so there is no need to store both. We only store for every positive literal to which literal it is equivalent and handle negations in `find`.

To identify rule applications that lead to some changes the `union` operation reports that, additionally `union` reports if it resulted in some inconsistency, so no further rules have to be applied in that recursion step.

We need to merge multiple equivalence relations, which can be done naively in $O(n^2)$. As the splitting and merging is structured with a dilemma calculus, we annotated the union-find with versions fields, so we can find the m modifications since the split and do the merging in $O(n + m^2)$.

References

[1] Magnus Björk. "First Order Stålmarck". en. In: *Journal of Automated Reasoning* 42.1 (Jan. 2009), pp. 99–122. ISSN: 1573-0670. DOI: 10.1007/s10817-008-9115-4. URL: https://doi.org/10.1007/s10817-008-9115-4.

[2] Mary Sheeran and Gunnar Stålmarck. "A Tutorial on Stålmarck's Proof Procedure for Propositional Logic". en. In: *Formal Methods in System Design* 16.1 (Jan. 2000), pp. 23–58. ISSN: 1572-8102. DOI: 10.1023/A:1008725524946. URL: https://doi.org/10.1023/A:1008725524946.

[3] Aditya Thakur and Thomas Reps. "A Generalization of Stålmarck's Method". en. In: *Static Analysis*. Ed. by Antoine Miné and David Schmidt. Lecture Notes in Computer Science. Springer, 2012, pp. 334–351. ISBN: 978-3-642-33125-1. DOI: 10.1007/978-3-642-33125-1_23.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

HKIS, HCAD, PRSPAKISINC and SBVAPRSPAKISINC in the SAT Competition 2025

Rodrigue Konan Tchinda[1], Clémentin Tayou Djamegni[2]
[1]*University of Bamenda, Bamenda, Cameroon*
[2]*University of Dschang, Dschang, Cameroon*
{rodriguekonanktr, dtayou}@gmail.com

***Abstract*—This document describes the sequential solvers HKIS and HCAD as well as the parallel solvers PRSPAKISINC and SBVAPRSPAKISINC submitted to the SAT Competition 2025.**

I. INTRODUCTION

Modern SAT solvers employ advanced techniques for performance, but even simple heuristics can yield significant gains. This is the case of the PSIDS (Polarity State Independent Decaying Sum) heuristic [1] that enhances efficiency by dynamically selecting variable polarities based on activity. While this method improves solver performance on unsatisfiable instances, it is worth noting that it may reduce effectiveness for satisfiable ones.

Preprocessing also plays a critical role in SAT solving. Notably, Structured Bounded Variable Addition (SBVA) [2] proved robust against formula shuffling. The latter, when integrated into solvers, consistently boost performance, as evidenced by the recent 2023 and 2024 SAT Competitions winners [3], [5], [6]. Our proposed solvers combine both PSIDS and SBVA. By integrating SBVA with PSIDS, we aim to optimize handling of complex unsatisfiable instances. This combination has shown promising results, as witnessed by the various prizes won in past competitions [1], [5], [7].

II. HKIS AND HCAD

For the 2025 SAT Competition, we submitted two versions of the solver HKIS. The first named HKIS2 updates its base solver to the version of Kissat that participated in the 2024 edition, while the second is identical to our 2024 submission. Likewise, HCAD remains unchanged from its 2024 version.

The configurations used are provided in Table I:

TABLE I

Config	Solvers	SBVA	Options
1	HKIS2	Yes	`psids=1 target=1 walkinitially=true chrono=true`
2	HKIS	Yes	`target=1 walkinitially=true chrono=true`
3	HCAD	Yes	`target=0 walk=false`
4	HCAD	Yes	`psids=1 target=2 walk=false chrono=true`

III. PRSPAKISINC AND SBVAPRSPAKISINC

The parallel solvers we submitted this year namely PRSPAKISINC and SBVAPRSPAKISINC [5] are the same as those submitted to SAT competition 2024, except that the number of workers has been increased to 28, and the preprocessing time for SBVA in SBVAPRSPAKISINC has been reduced by half.

IV. ACKNOWLEDGMENTS

We would like to thank the developers of KISSAT, CADICAL [8], KISSAT-INC [9], SBVA-KISSAT, SBVA-CADICAL [3] and PRS [4].

REFERENCES

[1] R. Konan Tchinda and C. Tayou Djamegni, "HKIS, HCAD, PAKIS and PAINLESS_ExMapleLCMDistChronoBT in the SC21," *SAT COMPETITION 2021*, p. 26, 2021.
[2] H. G. Haberlandt, Andrew and M. J. Heule, "Efective auxiliary variables via structured reencoding," in *International Conference on Theory and Applications of Satisfiability Testing*. Springer, 2023, pp. 11:1–11:19.
[3] A. Haberlandt, , and H. Green, "Sbva-cadical and sbva-kissat: Structured bounded variable addition," in *Proc. of SAT Competition 2023 – Solver and Benchmark Descriptions*, ser. Department of Computer Series of Publications B, M. I. M. J. Toms Balyo, Marijn J. H. Heule and M. Suda, Eds., vol. B-2023-1. University of Helsinki, 2023, p. 18.
[4] Y. Q. Zhihan Chen, Xindi Zhang and S. Cai, "Prs: A new parallel/distributed framework for sat," in *Proc. of SAT Competition 2023 – Solver and Benchmark Descriptions*, ser. Department of Computer Series of Publications B, M. I. M. J. Toms Balyo, Marijn J. H. Heule and M. Suda, Eds., vol. B-2023-1. University of Helsinki, 2023, p. 39.
[5] R. Konan Tchinda and C. Tayou Djamegni, "Hkis, hcad, prspakisinc and sbvaprspakisinc in the sat competition 2024," *SAT COMPETITION 2024*, p. 26.
[6] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "Cadical, gimsatul, isasat and kissat entering the sat competition 2024," *Proc. of SAT Competition*, pp. 8–10, 2024.
[7] R. Konan Tchinda and C. Tayou Djamegni, "HKIS, HCAD and PAKIS in the SAT Competition," in *Proc. of SAT Competition 2022 – Solver and Benchmark Descriptions*, ser. Department of Computer Science Series of Publications B, M. I. M. J. Toms Balyo, Marijn J. H. Heule and M. Suda, Eds., vol. B-2022-1. University of Helsinki, 2022, p. 24.
[8] A. Biere, K. Fazekas, M. Fleury, and M. Heisinger, "CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020," in *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, ser. Department of Computer Science Report Series B, T. Balyo, N. Froleyks, M. Heule, M. Iser, M. Järvisalo, and M. Suda, Eds., vol. B-2020-1. University of Helsinki, 2020, pp. 51–53.
[9] X. Z. S. C. Chen, Zhihan and P. Lu., "CDCL Solvers with Improved Local Search Cooperation and Pre-processing." *SAT COMPETITION 2022*, p. 37, 2022.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

DynamicCaDiCaL

Junjie Xu
Ningbo University
Ningbo, China
xjj375227926@gmail.com

***Abstract*—Traditional SAT solvers rely on fixed heuristic strategies throughout the solving process, often leading to performance degradation when facing diverse or dynamic instances. In this work, we present DynamicCaDiCaL, a self-adaptive extension of the CaDiCaL SAT solver. It incorporates a dynamic parameter tuning mechanism driven by a multi-armed bandit (MAB) algorithm, enabling real-time adaptation of heuristic strategies such as restart schemes, variable selection heuristics, and activity decay rates. Experimental observations indicate that this approach significantly improves average solving performance across heterogeneous SAT benchmarks, demonstrating the efficacy of adaptive heuristics guided by lightweight online learning.**

***Index Terms*—SAT solving, CaDiCaL, dynamic heuristics, multi-armed bandit, self-adaptive solver, parameter tuning.**

I. Introduction

Conflict-driven clause learning (CDCL) SAT solvers are the foundation of modern automated reasoning, widely adopted in formal verification, AI planning, and electronic design automation. While significant progress has been made in heuristics design—such as variable selection and restart strategies—most solvers adopt fixed parameter configurations. However, fixed heuristics may fall short when solving instances with varying structural or temporal characteristics.

To overcome this limitation, we propose DynamicCaDiCaL, an adaptive SAT solver that augments the open-source CaDiCaL [1] framework with two enhancements: (1) a dynamic parameter interface that allows the solver to modify its configuration at runtime, and (2) a multi-armed bandit (MAB) controller that selects configurations based on empirical performance. These innovations enable the solver to evolve its behavior over time, reducing the need for manual tuning and improving generality.

II. Related Work

Previous approaches to SAT solver tuning include offline parameter configuration tools (e.g., ParamILS, SMAC) and adaptive restarts. Kissat introduced simplified string-based configuration, while the MapleSAT family explored online learning for phase selection. DynamicCaDiCaL differs by combining multiple online heuristic dimensions and using a lightweight reward-based selection model during solving, without external supervision.

III. Dynamic Configuration Mechanism in DynamicCaDiCaL

A. Motivation

Heuristic effectiveness can vary significantly across benchmarks. A static strategy might be ideal for industrial SAT instances but inefficient for random ones. DynamicCaDiCaL dynamically reconfigures itself by collecting statistics during solving and selecting new strategies accordingly.

B. Dynamic Parameter Tuning

The core logic is embedded in the decide() function of CaDiCaL. Every fixed number of decisions, runtime indicators are collected, such as:

- Recent conflict and propagation counts
- Clause database size and simplification rate
- Average decision depth and restart interval

Based on these indicators, one of several pre-defined solver configurations is selected and applied via a Kissat-style string interface.

C. Strategy Selection via Multi-Armed Bandit

A multi-armed bandit (MAB) algorithm is used to choose among available strategies. Each "arm" corresponds to a parameter set such as:

The solver tracks the reward for each arm (e.g., conflict resolution rate), and uses an MAB policy (e.g., UCB1) to balance exploration and exploitation. Poorly performing strategies are de-prioritized over time, enabling a form of self-learning.

Recent work has also explored hybrid strategies to switch between heuristics based on restart events. For instance, Cherif et al.feedback. [2] proposed combining VSIDS and CHB branching heuristics by alternating them at restart points, using static and reinforcement learning methods. Their results demonstrated that dynamic switching can outperform the use of either heuristic alone in certain benchmark categories. Inspired by this direction, our MAB-based controller extends the idea by not only alternating between variable selection heuristics, but also adjusting a wider range of parameters, such as decay rates and rephase intervals, based on observed runtime feedback. [2]

D. Implementation Details

The implementation is focused on:

- decide.cpp, dynamicsat.cpp, dynanmicsat.hpp

- A module for collecting runtime statistics
- An MAB scheduler that updates and applies parameter strings
- Minimal overhead added to the decision process

E. Conclusion

This paper presented DynamicCaDiCaL, a novel SAT solver that integrates dynamic heuristic configuration with MAB-based strategy selection. By continuously adapting its internal parameters during solving, it improves performance without requiring problem-specific tuning. Future work includes extending the MAB model to incorporate contextual bandits or reinforcement learning agents for more fine-grained adaptation

REFERENCES

[1] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "Cadical 2.0," in *International Conference on Computer Aided Verification*, pp. 133–152, Springer, 2024.

[2] M. S. Cherif, D. Habet, and C. Terrioux, "Combining vsids and chb using restarts in sat," in *27th International Conference on Principles and Practice of Constraint Programming*, 2021.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Activity-Guided Dynamic BVA for amSAT

Chuang Zeng
Huazhong Univ. of Sci.&Tech.
Wuhan, China
zengzeng@hust.edu.cn

Mao Luo
Hubei Univ. of Tech.
Wuhan, China
luomao@hbut.edu.cn

Chu-Min Li
Picardie Univ. Jules Verne
Amiens, France
chu-min.li@u-picardie.fr

Shuolin Li
Aix Marseille Univ, Universite de Toulon
CNRS, LIS, France
shuolin.li@univ-amu.fr

Yiwen Huang
Huazhong Univ. of Sci.&Tech.
Wuhan, China
huangyw@hust.edu.cn

Zhipeng Lü
Huazhong Univ. of Sci.&Tech.
Wuhan, China
zhipeng.lv@hust.edu.cn

***Abstract*—We extend the preprocessing-based BVA (Bounded Variable Addition) [1] simplification technique to dynamically simplify learned clauses during the search process, guided by variable activity for literal selection.**

I. Introduction

Modern SAT solvers generate massive learned clauses through Conflict-Driven Clause Learning (CDCL), but rapid growth of the clause database leads to memory pressure and degraded propagation efficiency. This paper proposes a simplification method that integrates BVA with restart strategies, triggering BVA-based clause simplification during restart phases and guiding literal selection through variable activity. The core innovations include:

1) Dynamically detecting clause inflation and efficiency decline during restarts to adaptively activate BVA.
2) Combining variable activity with clause structural features to prioritize merging high-value variables.

The proposed method has been implemented in enhanced amSAT solvers, with effectiveness validated at SAT Competition 2025 [2, 3, 4].

II. Main Techniques

A. Dynamic Restart-Triggered Mechanism

Traditional BVA strategies are typically applied during preprocessing and struggle to adapt to dynamic search processes. Our approach synergizes restart strategies with BVA simplification:

1) Post-restart environments are "cleaned" by resetting all actively assigned variables to unassigned states and clearing temporary conflict clause traces, eliminating interference from obsolete data.
2) A historical reward-based trigger dynamically adjusts BVA activation timing: If previous simplifications yield limited benefits, subsequent BVA intervals are extended; conversely, intervals are shortened when significant gains are observed.

B. Activity-Guided Literal Selection

While conventional BVA employs a 3-hop strategy to break "grid" structures (focusing on variables within 3 hops of conflict variables to exploit local structures and avoid global traversal costs), our method leverages search-generated metadata:

1) Variable activity is incremented each time a variable participates in conflict analysis. High activity indicates recent frequent usage in conflict clauses, whereas low activity reflects long-term dormancy.
2) When two variables exhibit significantly divergent activity levels, the resolvent clauses generated through resolution operations may disrupt the CDCL search process. To mitigate this, we employ activity-guided literal selection to decouple variables with large activity discrepancies, thereby yielding superior solver performance.

III. Availability

The source code is available at https : https://github.com/wolfxi-5/amSAT_BVA.git

We would like to thank everyone who contributed to amSAT for their efforts and for making their tools publicly available.

References

[1] Haberlandt, H. Green, and M. J. H. Heule, "Effective auxiliary variables via structured reencoding," in 26th International Conference on Theory and Applications of Satisfiability Testing, SAT 2023, July 4-8, 2023, Alghero, Italy, ser. LIPIcs, M. Mahajan and F. Slivovsky, Eds., vol. 271. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, pp. 11:1–11:19.

[2] A. Haberlandt and H. Green, "SBVA-CADICAL and SBVAKISSAT: Structured bounded variable addition," in Proc. of SAT Competition 2023 – Solver and Benchmark Descriptions, ser. Department of Computer Science Report Series B, T. Balyo, N. Froleyks, M. J. H. Heule, M. Iser, M. Järvisalo, and M. Suda, Eds., vol. B-2023-1. University of Helsinki, 2023, p. 18.

[3] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "CaDiCaL 2.0," in Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I, ser. Lecture Notes in Computer Science, A. Gurfinkel and V. Ganesh, Eds., vol. 14681. Springer, 2024, pp. 133–152.

[4] K. Fazekas, F. Pollitt, M. Fleury, and A. Biere, "Certifying incremental SAT solving," in LPAR 2024: Proceedings of 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning, Port Louis, Mauritius, May 26-31, 2024, ser. EPiC Series in Computing, N. S. Bjørner, M. Heule, and A. Voronkov, Eds., vol. 100. EasyChair, 2024, pp. 321–340.

BENCHMARK DESCRIPTIONS

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Hard Combinatorial Instances with Symmetry

Markus Anders
RPTU Kaiserslautern-Landau
Kaiserslautern, Germany
0009-0004-5992-8433

Bart Bogaerts
KU Leuven
Leuven, Belgium
0000-0003-3460-4251

Benjamin Bogø
University of Copenhagen
Copenhagen, Denmark
0009-0000-2192-3291

Arthur Gontier
Glasgow University
Glasgow, United Kingdom
0000-0002-9579-5584

Wietze Koops
Lund University
Lund, Sweden
0000-0001-9945-1992

Jakob Nordström
University of Copenhagen & Lund University
Copenhagen, Denmark & Lund, Sweden
0000-0002-2700-4285

Andy Oertel
Lund University
Lund, Sweden
0000-0001-9783-6768

***Abstract*—We submit CNF formulas from two families, namely upper bounds for *Ramsey numbers* and *clique colouring* formulas. Both of these families contain non-trivial symmetry and are known to benefit from symmetry breaking techniques.**

I. Introduction

We submit CNF formulas which are known to benefit from symmetry breaking techniques. In particular, we submit instances from two families, namely upper bounds for *Ramsey numbers* and *clique colouring* formulas. Instances from both of these families contain *Johnson symmetry*, which is exploited by our solver submission SATSUMA+KISSAT.

All of our submitted instances were generated using the tool CNFGEN [1].

II. Ramsey Numbers

Ramsey's theorem [2] states that for any $r, s \in \mathbb{N}$ there exists the smallest positive integer $\mathrm{R}(r, s)$ (the so-called Ramsey number), such that any graph on $n \geq \mathrm{R}(r, s)$ vertices has to contain an r-clique or an independent set of size s.

The generated instances are satisfiable if and only if there exists a graph with n vertices which neither has a clique of size r, nor an independent set of size s. Hence, the instance is satisfiable if and only if $n < \mathrm{R}(r, s)$. Our submitted instances are generated to be *unsatisfiable* by considering known Ramsey numbers [3] and setting $n \geq \mathrm{R}(r, s)$.

The instance contains variables x_{v_1,v_2} for $1 \leq v_1 < v_2 \leq n$ encoding the edges of the graph, so $\binom{n}{2}$ variables in total.

We then encode that every subset of vertices of size r must not be a clique, meaning it contains at least one non-edge:

$$\bigwedge_{\substack{U \subseteq [n] \\ |U|=r}} \left(\bigvee_{\substack{u_1,u_2 \in U \\ u_1 < u_2}} \neg x_{u_1,u_2} \right).$$

We encode that every subset of vertices of size s must not be an independent set, meaning it contains at least one edge:

$$\bigwedge_{\substack{U \subseteq [n] \\ |U|=s}} \left(\bigvee_{\substack{u_1,u_2 \in U \\ u_1 < u_2}} x_{u_1,u_2} \right).$$

Hence, the instance has $\binom{n}{r} + \binom{n}{s}$ constraints.

We have tuned the parameters of our submitted instances, so that they can be efficiently solved using a SAT solver with symmetry breaking. The Ramsey number instances in our benchmark set have file names with the prefix `ramsey`.

III. Clique Colouring

The *clique-colouring formula* with parameters n, k, and c encodes the claim that there exists a graph on n vertices which has a clique of size k and is simultaneously c-colourable. This is a contradiction if $k > c$. In our benchmark instances we fix $k = c + 1$, so that all submitted instances are *unsatisfiable*.

Pudlák [4] proved that if $k = c + 1$ scales like $\sqrt[4]{n}$, then clique-colouring formulas are exponentially hard not only for resolution but even for the exponentially stronger cutting planes proof system [5]. However, if symmetry reasoning is used, then we can argue that without loss of generality there is a k-clique on the first k vertices, after which a second symmetry argument says that these k vertices have to be coloured with colours $1, 2, \ldots, c$ until we run out of colours.

Since Pudlák's result only provides asymptotic lower bounds, we have tuned $k = c + 1$ with respect to n to get suitably small formulas that should be hard for SAT solvers if they do not reason about symmetries. The clique-colouring formula instances in our benchmark set have file names with the prefix `clqcl`.

References

[1] M. Lauria, J. Elffers, J. Nordström, and M. Vinyals, "CNFgen: A generator of crafted benchmarks," in *Proceedings of the 20th International Conference on Theory and Applications of Satisfiability Testing (SAT '17)*, ser. Lecture Notes in Computer Science, vol. 10491. Springer, Aug. 2017, pp. 464–473.

[2] F. Ramsey, "On a problem of formal logic," *Proceedings of the London Mathematical Society*, vol. 2, no. 1, pp. 264–286, 1930.

[3] S. Radziszowski, "Small Ramsey numbers," *The Electronic Journal of Combinatorics*, Jun. 2024, Dynamic Survey DS1, Version 17.

[4] P. Pudlák, "Lower bounds for resolution and cutting plane proofs and monotone computations," *Journal of Symbolic Logic*, vol. 62, no. 3, pp. 981–998, Sep. 1997.

[5] W. Cook, C. R. Coullard, and G. Turán, "On the complexity of cutting-plane proofs," *Discrete Applied Mathematics*, vol. 18, no. 1, pp. 25–38, Nov. 1987.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Split-BDD Multiplier Verification Benchmarks Submitted to the SAT Competition 2025

Armin Biere*, Alexander Konrad*, Daniela Kaufmann†, Christoph Scholl*
* *University of Freiburg*, Germany † *TU Wien*, Austria

Formal verification of arithmetic circuits is an important problem but industrial practice still requires a substantial amount of manual effort. Recently an approach based on automated reasoning with polynomials using computer-algebra promises to scale much better than previous automated approaches, but still has issues with heavily optimized circuits.

In this context the verification of integer multiplier circuits serves as litmus test for the viability of such an approach. This is also our target here. For an overview on recent results and further pointers on integer multiplier verification see [1]–[4].

One idea is to use plain old reduced order *bit-level* binary decision diagrams (BDDs) for verifying integer multipliers, which, at first sight, looks impossible as building BDDs for integer multipliers is well known to explode [5]. However already Burch showed in 1991 [6], that splitting variables of a BDD, i.e., using new copies for each output slice, yields polynomial "split" BDDs. This idea has recently been revisited in [7]. The authors also give an algorithm for efficiently constructing a split-BDD specification for a given bit-width, which then can serve as *golden model* for an implementation.

The down-side of this approach we follow, is, first, that building the BDD for a given implementation, referred to as *symbolic simulation*, might still explode and, second, that the resulting split-BDD is not canonical and needs to be compared to the golden split-BDD. For this last task we build the XORs of BDDs of individual simulated and golden split-BDDs and then use a SAT solver to show that all of theses are *false*.

Thus our benchmarks encode the problem of showing the equivalence of the result of symbolic simulation of a gate-level optimized integer multiplier circuit to the golden split-BDD.

Using the GENMUL multiplier generator [8] we generated 252 Verilog models for 28 multiplier architectures for 9 bit-widths, from 16 to 48 with an increment of 4. Those were then synthesized with Yosys [9] into AIGs and further optimized with ABC [10] by applying the synthesis script "`resyn; resyn2; resyn3`" until fix-point.

Our internal prototype verification tool AIGAINDDS implements a variant of the approach presented in [7] and follows the procedure outlined above. At the end it yields a CNF for the resulting miter. In the CNF encoding we use 6 clauses for each node of the BDD to increase arc-consistency.

For specific multiplier architectures and larger bit-widths our symbolic simulation explodes despite splitting variables. We therefore had to limit main memory to 60 GB and in total produced 128 CNFs. However, it turned out that partial normalization inherent in the split-BDD approach produced many identical intermediate split-BDDs as well as CNFs and only 27 unique CNFs remain: 7 for bit-width 16, 6 for 20, 4 for 24, 3 for 28 and 32, and 1 for each of 36, 40, 44, 48. The benchmarks are named as "gm *n* sp *at fa*.cnf", with *n* the bit-width, *at* the architecture of the adder tree and *fa* of the final stage adder, e.g., "gm48sparrc.cnf" is a CNF for an architecture (array adder tree + ripple-carry final stage adder) for which we were able to produce CNFs for all bit-widths.

We evaluated these benchmarks with our award winning SAT solver KISSAT [11] and MINISAT. From the 27 benchmarks submitted both solved the same 10 benchmarks within one hour, with MINISAT being slightly faster.

This work was supported in part by the state of Baden-Württemberg through bwHPC, the German Research Foundation (DFG) through grant INST 35/1597-1 FUGG.

REFERENCES

[1] A. Mahzoon, D. Große, and R. Drechsler, "Revsca-2.0: Sca-based formal verification of nontrivial multipliers using reverse engineering and local vanishing removal," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 41, no. 5, pp. 1573–1586, 2022.

[2] D. Kaufmann and A. Biere, "Improving AMulet2 for verifying multiplier circuits using SAT solving and computer algebra," *Int. J. Softw. Tools Technol. Transf.*, vol. 25, no. 2, pp. 133–144, 2023.

[3] A. Konrad and C. Scholl, "Symbolic computer algebra for multipliers revisited - it's all about orders and phases," in *Formal Methods in Computer-Aided Design (FMCAD'24)*. IEEE, 2024, pp. 261–271.

[4] R. Li, L. Li, H. Yu, M. Fujita, W. Jiang, and Y. Ha, "RefSCAT: formal verification of logic-optimized multipliers via automated reference multiplier generation and SCA-SAT synergy," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 44, no. 2, pp. 791–804, 2025.

[5] R. E. Bryant, "Graph-based algorithms for boolean function manipulation," *IEEE Trans. Computers*, vol. 35, no. 8, pp. 677–691, 1986.

[6] J. R. Burch, "Using BDDs to verify multipliers," in *Proc. 28th Design Automation Conference (DAC'91)*. ACM, 1991, pp. 408–412.

[7] J. Kumar, Y. Miyasaka, A. Srivastava, and M. Fujita, "Formal verification of integer multiplier circuits using binary decision diagrams," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 42, no. 4, pp. 1365–1378, 2023.

[8] A. Mahzoon, D. Große, and R. Drechsler, *GenMul: Generating Architecturally Complex Multipliers to Challenge Formal Verification Tools*. Springer International Publishing, 2021, pp. 177–191.

[9] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanovic, "Yosys+nextpnr: An open source framework from Verilog to Bitstream for commercial FPGAs," in *27th IEEE Annual International Symposium on Field-Programmable Custom Computing Machines, (FCCM'19)*. IEEE, 2019, pp. 1–4.

[10] R. K. Brayton and A. Mishchenko, "ABC: an academic industrial-strength verification tool," in *Computer Aided Verification (CAV'10)*, ser. LNCS, vol. 6174. Springer, 2010, pp. 24–40.

[11] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024," in *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*, ser. Department of Computer Science Report Series B, vol. B-2024-1. University of Helsinki, 2024, pp. 8–10.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Bit-Level and Global Logical Equivalence Checking of Hardware Multiplier Architectures

Yalun Cai*†, Mengxia Tao*, Kezhi Li*†, Jiaying Zhu*†, Zhengyuan Shi*†, Qiang Xu*†
*National Center of Technology Innovation for EDA, Nanjing, China
†Department of Computer Science and Engineering, The Chinese University of Hong Kong, Sha Tin, Hong Kong S.A.R.

***Abstract*—This document describes the benchmarks that we submitted to SAT Competition 2025. We present a collection of logical equivalence checking problems encoded as CNF instances. These instances are derived from miters constructed using various 8-bit and 16-bit hardware multiplier architectures. Our benchmarks provide challenging yet practically relevant verification cases that evaluate a solver's ability to determine functional equivalence between different multiplier implementations.**

I. Introduction

Logical Equivalence Checking (LEC) [1] is a core technology in the field of hardware verification, mainly used to determine whether two circuit designs are completely equivalent in function. In the process of integrated circuit design, designers often need to optimize or transform the circuit without changing its functions, such as logic optimization, layout and routing, or technical mapping, etc. LEC technology can strictly prove whether these transformations maintain the original function of the circuit. As one of the significant applications of SAT solvers, LEC problems have received extensive attention in the SAT research community. Among the benchmarks submitted to SAT Competition 2024 [2], several originated from LEC problems, particularly those concerning the logical equivalence verification of multipliers. For SAT Competition 2025, we contribute new benchmarks derived from multiplier-based LEC problems. Although LEC for multipliers presents significant computational challenges for modern CDCL SAT solvers, and specialized verification techniques have demonstrated considerable efficiency in addressing these problems, investigating SAT-based approaches for such instances remains valuable. These challenging benchmarks can drive algorithmic innovations and theoretical advancements within the SAT research community.

II. Benchmark Generation

Our benchmark set can be divided into two categories: one assesses the logical equivalence across all outputs of two multiplier implementations, while the other examines the logical equivalence of specific individual output bits between multiplier implementations. We utilize the AIGER toolset [3] to construct miters and convert the resulting AIG files into CNF formulas. For bit-specific verification instances, we employ ABC [4] to extract the logical cones of specific output bits from the multiplier circuits prior to miter construction.

A. Multiplier Architectures

The benchmarks incorporate several distinct multiplier architectures, each with different implementation variants. These include Booth multipliers with Dadda or Wallace tree reduction, as well as standard array multipliers with Wallace or Dadda reduction trees. Each architecture offers unique characteristics in terms of circuit complexity, logic depth, and structural organization. The implementation variants include original (unoptimized) designs and various technology-mapped versions that represent different optimization strategies commonly employed in hardware synthesis.

B. Global Equivalence Checking

For global verification instances, we construct miters that compare the complete functional behavior of two multiplier implementations. These instances connect corresponding inputs of both multipliers and use XOR gates to compare all output bits simultaneously. The resulting miter circuit outputs a logical '1' if any output bit differs between the two implementations, thus forming a satisfiability problem where UNSAT indicates equivalence. These global instances generally present significant computational challenges due to the complexity of multiplier verification, particularly for the 16-bit implementations.

C. Bit-specific Equivalence Checking

The bit-specific verification instances focus on individual output bits, particularly targeting the higher-order bits (positions 22-31 for 16-bit multipliers) where verification complexity typically increases due to accumulated carry propagation effects. For these instances, we first extract the logical cone of the target output bit from each multiplier using ABC's cone extraction capabilities. This process significantly reduces the verification complexity while preserving the essential logic required to determine bit-specific equivalence. The resulting single-output circuits are then used to construct focused miters that verify equivalence solely for the specified output bit.

D. Benchmark Naming

The nomenclature of our benchmark instances follows a structured format that encodes the verification task parameters. Each filename includes the bit-width of the multipliers, followed by the architecture and variant of the first multiplier, then the architecture and variant of the second multiplier. For bit-specific verification instances, the filename additionally

contains a suffix indicating the specific output bit position being examined. The absence of this suffix denotes a global verification instance that checks equivalence across all output bits.

REFERENCES

[1] Andreas Kuehlmann and Florian Krohm. Equivalence checking using cuts and heaps. In *Proceedings of the 34th Annual Design Automation Conference (DAC '97)*, pages 263–268, 1997.

[2] Marijn J. H. Heule, Markus Iser, Matti Järvisalo, et al. Proceedings of SAT competition 2024: Solver, benchmark and proof checker descriptions. 2024.

[3] Armin Biere. Aiger (aiger is a format, library and set of utilities for and-inverter graphs (aigs)). https://fmv.jku.at/aiger/, 2006.

[4] Alan Mishchenko et al. Abc: A system for sequential synthesis and verification. *URL http://www. eecs. berkeley. edu/alanmi/abc*, 17, 2007.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Benchmarks from equivalence checking of MD5 circuits

Shuang Chen*
College of Computer Science and Technology
National University of Defense Technology
Changsha,China
chenshuang21a@nudt.edu.cn

Jianmin Zhang*
College of Computer Science and Technology
National University of Defense Technology
Changsha,China
jmzhang@nudt.edu.cn

Yan Sun
College of Computer Science and Technology
National University of Defense Technology
Changsha,China
yansun@nudt.edu.cn

***Abstract*—Equivalence verification for RTL designs is one of the primary applications of SAT solvers. We generated 20 SAT instances through an open source EDA tool from sequential equivalence verification for MD5 circuits.**

I. INTRODUCTION

Formal equivalence checking [1] between an original Verilog/VHDL design and its optimized version, or its post-synthesis netlist, or its post-routing netlist constitutes a core use case for SAT-solving technologies. In circuit equivalence checking, we need to convert the logical relationships of the circuit into a Boolean formula and then use the SAT solver to judge whether there exists an input combination that makes the outputs of the two circuits different. After the SAT solver solves the CNF formula, it will return a result. If the formula is satisfiable, it means that there is a counterexample, that is, an input combination has been found to make the outputs of the two circuits different, indicating that the two circuits are not equivalent. If the formula is unsatisfiable, it means that no such counterexample exists, and the two circuits are functionally equivalent. This method has become a gold standard in functional verification for chip design and EDA toolchains.

In practical applications, complex situations such as combinational equivalence checking (CEC) and sequential equivalence checking (SEC) will be encountered. In CEC problems, since the correspondence between the states and input/output of the two circuits has been determined, it can be directly reduced to a comparison of binary decision diagrams or a SAT solving problem. In SEC problems, it is necessary to verify that the outputs of the two circuits are the same at each clock cycle under any legal input. This requires finding the correspondence between the internal states of the two circuits and may need to handle complex situations such as timing rearrangement. Modern equivalence verification tools will handle timing rearrangement information specially.

II. BENCHMARKS

Yosys [2] is an open-source EDA tool family, primarily used for RTL (Register Transfer Level) synthesis and formal verification. It supports various hardware description languages (such as Verilog and VHDL) and provides a complete toolchain for tasks like logic synthesis, SAT based equivalence checking, and model checking.

We have generated 20 CNF formulas by Yosys running the equivalence checking between the RTL design and its post-synthesis netlist or modified netlist. The –seq option in Yosys directs the sat command to analyze sequential circuits by solving satisfiability problems over specified time steps. We designed a kind of encryption and decryption application circuits and different time steps to generate CNF. The RTL designs implemented the parallel MD5 (Message-Digest v5) algorithm. In order to make the problems more difficult, we also consider to add certain bugs in the instances.

REFERENCES

[1] Andrade FV, Oliveira M, Fernandes A, Coelho C. SAT-Based equivalence checking based on circuit partitioning and special approaches for conflict clause reuse. 2007 IEEE Design and Diagnostics of Electronic Circuits and Systems, Krakow, Poland, 2007: 1-6.

[2] YosysHQ GmbH. https://github.com/YosysHQ/yosys.

*These authors contributed equally to this work.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Challenging Certificates from Model Checking

Nils Froleyks
Johannes Kepler University
Linz, Austria

Emily Yu
Institute of Science and Technology Austria
Klosterneuburg, Austria

Armin Biere
University of Freiburg
Freiburg, Germany

The Hardware Model Checking Competition 2024 [1] introduced certificates to the bit-level track of the competition in the form of *witness circuits* [2]. Checking the correctness of a witness circuit entails solving a set of five SAT formulas. If all of them are unsatisfiable the witness circuit is valid, and the safety of the original model is proven.

The model checkers participating in the bit-level track of the competition have to prove that all states a sequential Boolean circuit can reach meet a given property. A circuit $M = (I, L, R, F, P, C)$ is modeled as a set of non-deterministic inputs I and a set of latches L which are initialized using their *reset function* in R and change value according to their transition function in F. Finally, C is a constraint that can be used to restrict the states to be considered, and P is the property to proof.

We use the following notation to denote that the latches in L are in a reset state $R\{L\} = \bigwedge_{\ell \in L} \ell{=}r_\ell(I, L)$, or follow the transition function $F_{0,1}\{L\} = \bigwedge_{\ell \in L} \ell_1{=}f_\ell(I_0, L_0)$. In the latter we use a lower index to refer to multiple temporal copies, i.e., multiple states along a trace.

If a circuits violates the property P, the model checkers have to return a sequence of inputs satisfying:

$$R_0\{L\} \wedge \bigwedge_{i \in [0,n)} F_{i,i+1}\{L\} \wedge \bigwedge_{i \in [0,n]} C_i \wedge \neg P_n.$$

If the property holds they have to return a witness circuit $W = (I', L', R', F', P', C')$ which satisfies:

Reset:	$R\{K\} \wedge C \Rightarrow R'\{K\} \wedge C'$
Transition:	$F_{0,1}\{K\} \wedge C_0 \wedge C_1 \wedge C'_0 \Rightarrow F'_{0,1}\{K\} \wedge C'_1$
Property:	$(C \wedge C') \Rightarrow (P' \Rightarrow P)$
Base:	$R'\{L'\} \wedge C' \Rightarrow P'$
Step:	$P'_0 \wedge F'_{0,1}\{L'\} \wedge C'_0 \wedge C'_1 \Rightarrow P'_1$

Where $K = L \cap L'$ is the intersection between the circuits. Additionally, R' has to be stratified [3].

These checks are generated using CERTIFAIGER [4] and translated to CNF, thus yielding five formulas per model checking instance. Usually, all checks except for *Transition* and *Step* are trivial, as they do not encode the transition behavior, which usually is the most complex part.

Table list the 20 benchmarks submitted to the SAT competition. Since all of the witness circuits produced by RIC3 are correct, the submitted benchmarks only contain unsatisfiable benchmarks. Many of them could not be solved by MINISAT within a time limit of 10 hours. On the other hand, all 1492 valid certificates produced during the model checking competition where successfully checked within the same time limit using KISSAT 4.0. In fact, the overall certification overhead for the winning model checker RIC3 was only 34%.

Model	**rIC3**	**Check**	**miniSAT**
x-epic_a19-p15	0.64 s	Transition	$> 10h$
x-epic_a10-p53	1.28 s	Transition	438 s
cal182_cal182	3.22 s	Transition	$> 10h$
yosyshq_cv32e40x-p500	6.64 s	Transition	$> 10h$
yosyshq_cv32e40x-p749	6.74 s	Transition	$> 10h$
yosyshq_veer-p15	8.86 s	Transition	$> 10h$
yosyshq_axi-p23	9.88 s	Transition	34989 s
bv_ILA_Piccolo_JALR_sanity	13.14 s	Transition	26415 s
x-epic_a19-p16	13.89 s	Step	$> 10h$
x-epic_a19-p16	13.89 s	Transition	$> 10h$
bv_ILA_Piccolo_BEQ_sanity	17.20 s	Transition	27000 s
nla_freire1_valuebound1	17.81 s	Transition	$> 10h$
yosyshq_veer-p28	23.90 s	Transition	33821 s
bv_rocket_1951	32.62 s	Transition	766 s
yosyshq_axi-p06	35.16 s	Transition	33453 s
float-benchs_zonotope_2	37.98 s	Transition	394 s
2018D_VexRiscv-regch0-20-p1	150.54 s	Step	2788 s
nla_hard-ll_valuebound20	188.18 s	Transition	$> 10h$
nla_dijkstra-u_valuebound1	822.94 s	Step	$> 10h$
nla_dijkstra-u_valuebound1	822.94 s	Transition	$> 10h$

TABLE I
THE TABLE LIST BENCHMARKS SELECTED FROM THE HARDWARE MODEL CHECKING COMPETITION 2024 WITH THE TIME RIC3 TOOK TO PERFORM MODEL CHECKING, AND THE TIME MINISAT TOOK TO VALIDATE THE STATED CERTIFICATE VALIDATION CHECK.

The benchmark generator is available on Zenodo [5].

REFERENCES

[1] A. Biere, N. Froleyks, and M. Preiner, "Hardware model checking competition (hwmcc'20)," 2020. [Online]. Available: https://hwmcc.github.io/2020

[2] N. Froleyks, E. Yu, M. Preiner, A. Biere, and K. Heljanko, "Introducing certificates to the hardware model checking competition," in *Computer Aided Verification (CAV)*, 2025.

[3] E. Yu, N. Froleyks, A. Biere, and K. Heljanko, "Stratified certification for k-Induction," in *22nd Formal Methods in Computer-Aided Design, FMCAD 2022, Trento, Italy, October 17-21, 2022*, A. Griggio and N. Rungta, Eds. IEEE, 2022, pp. 59–64.

[4] N. Froleyks, E. Yu, and A. Biere, "Certifaiger on github," 2025. [Online]. Available: https://github.com/Froleyks/certifaiger

[5] N. Froleyks, E. Yu, and A. Biere, "Challenging certificates from model checking," Apr. 2025. [Online]. Available: https://doi.org/10.5281/zenodo.15267003

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

SAT Encodings of P-Center Problem

Shuolin Li*, Jialu Zhang†
*Aix Marseille Univ, Université de Toulon, CNRS, LIS, Marseille, France
shuolin.li@lis-lab.fr
†Université de Picardie Jules Verne, Amiens, France
jialu.zhang@u-picardie.fr

***Abstract*—This document is an introduction of the benchmarks submitted to the SAT Competition 2025.**

I. Introduction

We consider the problem of optimally locating charging stations for electric vehicles in the Bouches-du-Rhône region of France. Given a set of nodes N, each node $i \in N$ in the region represents a candidate location — which coincide with the demand node — with known coordinates and a number of electric vehicles requiring charging. An integer p representing the number of charging stations to install, the objective is to select p locations from N to install the stations. Each demand node must be assigned to its nearest installed station, and the goal is to minimize the maximum distance between any demand node and its assigned charging station.

This is essentially the p-center problem, a classic combinatorial optimization problem. It can be approached by iteratively solving a sequence of decision subproblems: for a given coverage radius r_i, we construct a graph by removing all edges (or distance links) longer than r_i. The decision problem then becomes: is it possible to cover all demand nodes using at most p facilities such that each node is within distance r_i of at least one facility?

To solve this using SAT, we follow the approach in [1], where the problem is first reduced to a Set Covering formulation and then encoded to CNF. For a fixed radius r_i, we introduce a Boolean variable y_i for each node i, indicating whether a charging station is placed there. Each demand node must be covered by at least one open station within radius r_i, and no more than p variables can be set to true.

This leads to the following ILP model:

$$\begin{aligned} &\sum_{i \in N : d_{ij} \le r_i} y_i \ge 1 \quad \forall j \in N \\ &\sum_{i \in N} y_i \le p \\ &y_i \in \{0,1\} \quad \forall i \in N \end{aligned} \tag{1}$$

The first constraint can be encoded to CNF as:

$$\bigvee_{i \in N : d_{ij} \le r_i} y_i, \forall j \in N$$

which ensures that every demand node is covered by at least one facility within the allowed radius.

The second constraint, which limits the number of selected centers to exactly p, is encoded to CNF using a sequential counter.

II. Benchmarks

The name of a benchmark file follows the pattern: $name$_thres$x$_py_rz.cnf, where $name$ denotes the city, x is the node threshold, y is the number of charging stations (p), and z is the coverage radius. Only nodes with more than x electric vehicles are included in the set N.

References

[1] X. Liu, Y. Fang, J. Chen, Z. Su, C. Li, and Z. Lü, "Effective approaches to solve p-center problem via set covering and sat," *IEEE Access*, vol. 8, pp. 161 232–161 244, 2020.

Acknowledgment

This work is supported by AI CHAIR reference ANR-19-CHIA-0013-01 (MASSAL'IA) funded by the French Agence Nationale de la Recherche and the electricity distribution network manager Enedis (project partner), and partially supported by Archimedes Institute, Aix-Marseille University. We want to thank the Université de Picardie Jules Verne for providing the Matrics Platform.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Benchmarks Encoding the Preferred Extension of Argument Framework

Qi Liu
Hubei University of Technology
Wuhan, China
102401137@hbut.edu.cn

Mao Luo
Hubei University of Technology
Wuhan, China
luomao@hbut.edu.cn

Chu-Min Li
MIS, University of Picardie Jules Verne
Amiens, France
chu-min.li@u-picardie.fr

Caiquan Xiong
Hubei University of Technology
Wuhan, China
xiongcq@hbut.edu.cn

***Abstract*—This document presents a set of benchmark instances encoding the SE-PR (Some Extension under Preferred Semantics) problem, derived from argumentation models. The SE-PR tasks are systematically translated into SAT instances in standard DIMACS CNF format. These benchmark instances are submitted for the SAT Competition 2025, aiming to evaluate SAT solvers on problems that combine combinatorial complexity with abstract argumentation reasoning.**

I. Introduction

Argumentation theory has found significant applications in automated reasoning, particularly in computing Some Extension under Preferred Semantics (SE-PR). This work focuses on encoding SE-PR tasks into SAT instances, derived from the International Competition on Computational Models of Argumentation (ICCMA) competition 2023 main track benchmark instances [1]. These instances are formulated in DIMACS CNF format and are designed to evaluate the performance of SAT solvers on complex argumentation reasoning problems.

We submit 20 benchmark instances in total, which are mainly divided into two categories: those prefixed with `"crusti_g2io"` and those prefixed with `"st"`.

- The `"crusti_g2io"` instances were generated using a benchmark generator based on an inner/outer template [2].
- The `"st"` instances were created by Matthias Thimm using his instance generator, originally developed for ICCMA 2015 [3].

II. Formal Description

A. Problem Definition

Let $\Gamma = \langle A, R \rangle$ be an argumentation framework (AF) as defined by Dung [4], where A is a finite set of arguments and $R \subseteq A \times A$ denotes the attack relation. A set $S \subseteq A$ is a *preferred extension* if it is a maximal admissible set.

B. Generation of preferred extensions

The computation of preferred extensions using ArgSemSAT [5] primarily follows the steps outlined below:

Labeling Constraints: Each argument must be assigned exactly one of the three mutually exclusive labels:

- `in` (I_i): the argument is accepted.
- `out` (O_i): the argument is rejected.
- `undec` (U_i): the argument is neither accepted nor rejected due to uncertainty.

The following logical constraint guarantee that each argument is assigned exactly one label among `in`, `out`, and `undec`:

$$\bigwedge_{i\in\{1,\ldots,k\}} \Big((I_i \vee O_i \vee U_i) \wedge (\neg I_i \vee \neg O_i) \wedge (\neg I_i \vee \neg U_i) \wedge (\neg O_i \vee \neg U_i)\Big)$$

Complete Labeling Constraints:

We systematically examine all arguments and apply the labeling process by considering their inter-attack relationships, using the constraints defined in the following formulas.This process ensures that arguments are appropriately classified based on the attack structure. Ultimately, the set of arguments labeled as `in` represents the complete extension of the argumentation model.

$$\bigwedge_{\{i|\phi(i)^- \neq \emptyset\}} \left(\bigwedge_{\{j|\phi(j)\to\phi(i)\}} \neg I_i \vee O_j \right)$$

$$\bigwedge_{\{i|\phi(i)^- \neq \emptyset\}} \left(\neg O_i \vee \left(\bigvee_{\{j|\phi(j)\to\phi(i)\}} I_j \right) \right)$$

$$\bigwedge_{\{i|\phi(i)^- \neq \emptyset\}} \left(\left(\bigwedge_{\{j|\phi(j)\to\phi(i)\}} (\neg U_i \vee \neg I_j) \right) \wedge \left(\neg U_i \vee \left(\bigvee_{\{j|\phi(j)\to\phi(i)\}} U_j \right) \right) \right)$$

Preferred Labeling Constraints:

The preferred labeling semantics identifies complete labelings in which the set of arguments labeled as `in` is maximized. This process consists of two main steps:

1) First, we examine the set of arguments labeled as `in`. If this set is empty, we directly return the empty set, indicating that the preferred extension for the argumentation model is the empty set.
2) Second, we iteratively attempt to include additional arguments into the `in` set. For each candidate argument, we check whether its inclusion—along with the propagation of labeling constraints—leads to a conflict. If no conflict arises, the argument is added to the `in` set, thereby expanding it. Otherwise, the process continues by examining the next candidate.

C. Retrieval of Benchmark Instances

During the process of solving the SE-PR problem, the argumentation problem is repeatedly transformed into CNF-format SAT instances, which are subsequently solved by a SAT solver. Throughout this procedure, we log each generated SAT instance along with its corresponding solving time. Based on the requirements of the SAT Competition 2025, we then select those instances that meet the designated criteria to serve as benchmark instances.

III. Instance testing

To ensure the benchmark instances meet the evaluation requirements, we tested the generated instances using both the Cadical and the Kissat SAT solver. The performance and compatibility of these instances were verified through solver execution, as summarized in the following table.

Instance Name	**Cadical Time (s)**	**Kissat Time (s)**
crusti_g2io_175_0.2_511_10.cnf	638.54	506.27
crusti_g2io_175_0.2_511_32.cnf	715.43	503.82
crusti_g2io_175_0.2_511_48.cnf	752.45	573.77
crusti_g2io_200_0.1_127_19.cnf	421.76	231.61
crusti_g2io_225_0.1_31_25.cnf	123.47	87.56
crusti_g2io_250_0.2_255_12.cnf	276.35	190.66
crusti_g2io_250_0.2_255_18.cnf	412.32	296.3
crusti_g2io_250_0.2_255_24.cnf	521.68	455.63
crusti_g2io_250_0.2_255_31.cnf	201.32	142.28
crusti_g2io_250_0.2_255_43.cnf	293.23	265.42
st_1352_53_23_3737.cnf	over 1800	over 5000
st_1391_70_12_1674.cnf	over 1800	over 5000
st_1400_85_28_2113.cnf	over 1800	over 5000
st_1412_95_15_3263.cnf	over 1800	over 5000
st_1489_41_39_1070.cnf	over 1800	over 5000
st_659_37_25_686.cnf	over 1800	over 5000
st_815_74_9_2860.cnf	over 1800	over 5000
st_826_34_8_3910.cnf	over 1800	over 5000
st_883_28_16_1144.cnf	over 1800	over 5000
st_890_86_9_572.cnf	over 1800	over 5000

TABLE I: Benchmark instances testing times

References

[1] M. Järvisalo, T. Lehtonen, and A. Niskanen, "ICCMA 2023: Benchmarks and raw results," Available: https://zenodo.org/records/8348039.

[2] J.-M. Lagniez, E. Lonca, J.-G. Mailly, and J. Rossit, "Crusti g2io: A Benchmark Generator Based on an Inner/Outer Template," in *Solver and Benchmark Descriptions of ICCMA 2023: 5th Int. Competition on Computational Models of Argumentation*, vol. B-2023-3, Dept. of Computer Science, Series of Publications B, Univ. of Helsinki, 2023, pp. 35–36.

[3] M. Thimm, "StableGenerator," Jul. 17, 2017. Available: https://sourceforge.net/p/probo/code/HEAD/tree/trunk/src/net/sf/probo/generators/StableGenerator.java

[4] P. M. Dung, "On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games," *Artif. Intell.*, vol. 77, no. 2, pp. 321–358, 1995. Available: https://doi.org/10.1016/0004-3702(94)00041-X

[5] F. Cerutti, M. Vallati, and M. Giacomin, "ArgSemSAT: Solving Argumentation Problems Using SAT," in *Proc. 5th Int. Conf. on Computational Models of Argument (COMMA 2014)*, Frontiers in Artificial Intelligence and Applications, vol. 266, IOS Press, 2014, pp. 455–456.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Combinational Equivalence Checking Instances

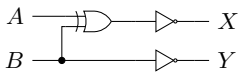
Chuan Luo, Zhenwei Yang, Shaoke Cui, Chunming Hu
School of Software
Beihang University
Beijing, China
{chuanluo, zwyang, cuisk, hucm}@buaa.edu.cn

I. Introduction

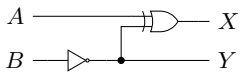
Combinational equivalence checking is one of the most important applications of SAT solvers. The benchmarks we proposed consist of the encoded isomorphic miters from circuits in the Hardware Model Checking Competition (HWMCC) 2015 [1]. All the benchmarks are UNSAT.

II. Benchmarks

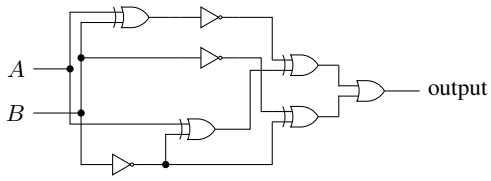
Circuits in HWMCC 2015 are used to generate the miters. Isomorphic circuits are obtained by applying optimizations on the original circuits. To check the equivalences between the original circuit and the optimized circuit, pairs of corresponding inputs are combined, and pairs of corresponding outputs are fed into extra XOR gates to check whether they are different. These XOR gates are then fed into an OR gate, which produces the single output of the miter. The miter has the same inputs as the original circuit, and outputs a constant 0 if and only if the two circuits are equivalent. Figure 1 shows an example of the miter of two combinationals in Figures 1a and 1b. The generated miter circuits are then encoded to the CNFs through an optimized Tseitin encoding [2], which is different from the related miter benchmarks.



(a) Combinational.



(b) Isomorphic combinational.



(c) Miter of two combinationals.

Fig. 1: An example of miter.

The instances were evaluated using the latest Kissat solver [3]. According to the evaluation results, 30 interesting instances were selected and submitted as benchmarks, which can be solved within an hour but took more than 10 minutes. All the benchmarks are UNSAT.

References

[1] Armin Biere and Keijo Heljanko, "Hardware Model Checking Competition 2015", 2015. Available: https://fmv.jku.at/hwmcc15/
[2] G. S. Tseitin, "On the complexity of derivation in propositional calculus", Studies in Constructive Mathematics and Mathematical Logic, Part II, Seminars in mathematics, pp. 115–125, 1970.
[3] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks and Florian Pollitt, "CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2024", Proc. SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions, vol. B-2024-1, pp. 8–10, 2024.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Odd Ball Weighing Problems

Andrew Mihal
Hobbyist
Email: andrewcmihal@gmail.com

***Abstract*—A classic problem in recreational mathematics is to identify an incorrectly-weighted ball in a set of N otherwise identical balls. The challenge is to use a balance scale a maximum of W times, where W is typically much smaller than N. The solution must identify which ball has the incorrect weight and whether the ball is heavier or lighter that the rest. We propose a collection of SAT benchmarks that solve this problem for various configurations.**

I. Problem Description

This brainteaser has appeared in the literature in numerous variations for many years. It is sometimes called the Odd Coin Problem or the Counterfeit Coin Problem. The specific variation addressed in this paper is described by Bundy [2]. Students are usually tasked with solving the problem first with 9 balls and 3 weighings, and then are challenged with the more complicated 12 ball, 3 weighing version. It is clear that the ternary nature of the weighing outcomes is critical to the solution.

We implement a SAT-based solution to this puzzle that generalizes to an arbitrary number of balls N and number of weighings W. Combined with various symmetry-breaking strategies, these puzzles create instances that are useful for benchmarking the performance of SAT solvers.

Constructive solutions to this puzzle are known, for example the solution due to Wert [3]. In this work we make the simplifying assumption that the balls to be weighed are determined *a priori*. That is, the choice of what to weigh next is not dependent on the outcome of the previous weighings. Wert's solution in comparison has decision branches.

Some variations provide a known-good ball, or provide known counterweights, or give away the information that the odd ball is lighter or heavier, or provide the specific delta weight between the good balls and the odd ball. We do not address these variations. We assume that the odd ball weight varies by only ϵ and therefore the same number of balls must be on both sides of the scale for every weighing.

Our solution determines the set of balls on the left and right sides of the scale for each of the W weighings. The outcomes of the weighings are then looked up in a truth table that tells which ball is wrong and whether it is light or heavy.

II. SAT Encoding

The Oddball program, available on GitHub [1], is implemented in Python using Z3Py. The formulation uses an $W \times N \times N$ array of Boolean variables B, where $B[w, n_1, n_2]$ indicates that ball n_1 is on the left side of the scale and ball n_2 is on the right side of the scale in weighing w. At-Most-1 cardinality constraints are applied to ensure that a ball does not appear on both sides of the scale at the same time, or appear more than once on the same side of the scale.

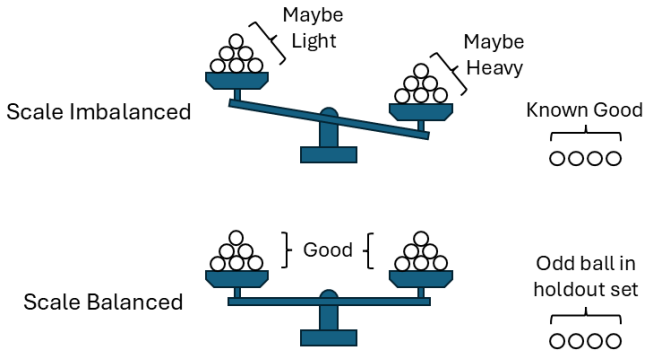


Fig. 1. Weighing Result Implications

A second set of Boolean variables T model the truth table. Variable $T[r, b, e]$ indicates that row $r \in [0, 3^W)$ has the outcome that ball $b \in [0, N)$ has error $e \in [-, +]$, meaning that ball b is light or heavy respectively. Each truth table row must imply at most one ball-error combination. Every ball-error combination must appear at least once in the truth table.

The outcome of each weighing implies possibilities about the location and error of the odd ball (see Figure 1). If the scale is balanced, we know that all of the balls on the scale are good and the odd ball must be in the holdout set that is not on the scale. If the scale is imbalanced, we know that the odd ball must be on the scale. It could be a light ball on the light side of the scale, or it could be a heavy ball on the heavy side of the scale. All the balls in the holdout group are known to be good.

For each truth table row r, the outcomes of the W weighings are known, and implications can be added to the formula between the B variables and the T variables for that row.

The goal of the SAT solver is to choose pairs of balls to weigh in each weighing such that each truth table row implies at most one result and all possible results appear at least once in the truth table.

The Python program can solve the formulation directly with Z3, or it can export CNF or SMT2 formats. It can also re-import certificates to print and verify solutions generated by an external solver.

III. Example Solution

The following example solution is for the $N = 12, W = 3$ configuration. The first section shows the sets of balls to place on the scale for each weighing. The second section gives the truth table that maps the weighing outcomes to ball-error results. The weighing outcome $<$ indicates that the left side of the scale is lighter than the right, $=$ means the scale is balanced, and $>$ means the left side is heavy.

```
Solution:
W0: [ 1,  2,  3,  6] <=> [ 0,  4,  5,  7]
W1: [ 1,  2,  7,  9] <=> [ 0,  6,  8, 10]
W2: [ 3,  5,  6, 10] <=> [ 0,  2,  8, 11]

Truth Table:
ix  W0  W1  W2   Result
 0   <   <   <   0+
 1   <   <   =   1-
 2   <   <   >   2-
 3   <   =   <   3-
 4   <   =   =   4+
 5   <   =   >   5+
 6   <   >   <   6-
 7   <   >   =   7+
 8   <   >   >
 9   =   <   <   8+
10   =   <   =   9-
11   =   <   >   10+
12   =   =   <   11+
13   =   =   =
14   =   =   >   11-
15   =   >   <   10-
16   =   >   =   9+
17   =   >   >   8-
18   >   <   <
19   >   <   =   7-
20   >   <   >   6+
21   >   =   <   5-
22   >   =   =   4-
23   >   =   >   3+
24   >   >   <   2+
25   >   >   =   1+
26   >   >   >   0-
```

IV. Symmetries

The problem has a few symmetries that can be exploited for better solver performance.

First, the problem is symmetric under permutations of the ball labels. This symmetry can be broken with an ordering constraint on the ball numbers as they appear in the truth table. Each row can imply no result (since the number of ball-error combinations may be less than 3^W), or the opposite error for a ball that has already appeared in the truth table, or it can imply an error for the sequentially next ball that has not yet appeared. This symmetry-breaking strategy was used for the example solution above.

An alternative that weakly breaks the label permutation symmetry is to enforce an ordering on the balls that are placed on the scale in the first weighing. Pair 0,1 must be placed on the left and right sides first, followed by pair 2,3, pair 3,4 and so forth. The solver is free to choose how many pairs to weigh in the first weighing but they must appear in this order. This strategy is weaker than the first because it still permits label permutations of the balls in the first holdout set. Nevertheless this is an option provided by the program to improve the variety of generated instances.

Second, the truth table itself has mirror symmetry top-to-bottom. This is a consequence of symmetry around the odd ball's error. Imagine that the odd ball is on the scale and the scale is therefore imbalanced. If the error on the ball is flipped, then the scale will also flip. In any truth table row, if all the $<$ and $>$ outcomes are reversed, then the result must be the same ball number but with the opposite error.

Note that the middle row of the truth table with all $=$ outcomes cannot imply a result. We know that the odd ball must be in the holdout set for every weighing. This can be enough information to uniquely determine the odd ball, but not whether it is lighter or heavier than the others.

The truth table mirror symmetry can be exploited by only solving for the top half of the truth table in the SAT formulation.

V. Benchmarks

Configurations with $N \approx 50$ and $W = 5$ can be solved by Kissat [5] in the 10 to 20 minute range. For our proposed benchmarks we generate examples around this point as well as some in the $N \approx 10$, $W = 6$ range. All are SAT.

For UNSAT instances, we could generate examples with $N > (3^W - 3)/2$ (the maximum solvable N for any W). The $N = 13$, $W = 3$ instance is trivial for Kissat and the $N = 40$, $W = 4$ instance takes more than one hour, so they are not suitable benchmarks. Instead, we use an intentionally broken version of the truth table mirror symmetry strategy to generate UNSAT instances that take Kissat up to 45 minutes to solve. The broken strategy forces the balls to appear sequentially in the top half of the truth table without any empty rows.

VI. Conclusion

This work was motivated while the author was helping math students with their homework during ski week in Austria. The $N = 12$, $W = 3$ problem was given as a bonus challenge with the implication that it might not be solvable. One student asked ChatGPT to solve the problem and ChatGPT said it was impossible. The student, predisposed to stopping work anyway, decided to believe the AI and gave up. This made the author upset.

While the approach used in this paper may not be optimal, it has been able to solve problems up to $N = 36$, $W = 6$ with 15 minutes of Kissat runtime.

References

[1] A. Mihal. "Oddball," 2025, GitHub repository. [Online]. Available: https://github.com/acmihal/oddball

[2] B. Bundy. "The Oddball Problem," 1996/7, Mathematical Spectrum, v 29, no 1, pg 14–15

[3] J. Wert. "Odd Coin Problems," [Online]. Available: https://cut-the-knot.org/blue/OddCoinProblems.shtml

[4] L. De Moura and N. Bjørner. "Z3: An Efficient SMT Solver," In TACAS'08. 337–340

[5] A. Biere et al. "CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2024". In Proc. SAT Comp. 2024. 8–10

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Bel's Pyramid Puzzle

Andrew Mihal
Hobbyist
Email: andrewcmihal@gmail.com

Belgacem (Bel) Haba
Puzzle Enthusiast
Email: all4dz@gmail.com

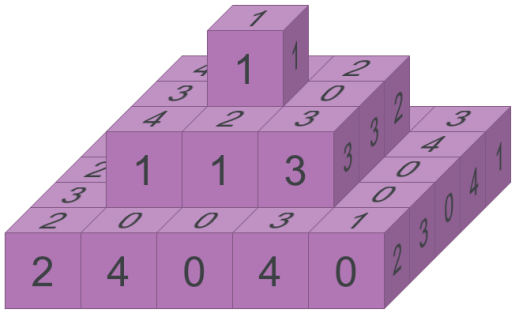


Fig. 1. Solved $N = 3$ Pyramid

***Abstract*—Bel's Pyramid is a recreational mathematics puzzle involving stacking cubes to create a square step pyramid, under the constraint that adjacent cube faces must have the same label. Solutions for pyramid sizes up to $N = 5$ are currently known. We propose a collection of SAT benchmarks that solve this puzzle for various configurations, including variations on the SAT encoding, different symmetry-breaking strategies, and proofs that some constructive solutions are infeasible.**

I. Problem Description

Bel's Pyramid is a cube-stacking puzzle invented by the co-author. The goal is to create a square step pyramid with N levels using a set of uniquely-labeled cubes. Cube faces that touch must have the same label. The cubes are labeled in a special way that is detailed below.

It is hypothesized that multiple solutions exist for every puzzle size N, but as of this writing there is no known constructive approach for generating any solution for an arbitrary N. All known solutions for non-trivial puzzles have been discovered using search (specifically SAT). SAT has also been able to prove that some proposed constructive approaches are infeasible. The end goal is to find a general solution.

Figure 1 shows the geometry of a solved pyramid with $N = 3$ layers. Each layer $n \in [0, N)$ is a square with side length $2n + 1$. The layers are aligned vertically along their center cubes to create a symmetric step pyramid. The total number of cubes in the pyramid is:

$$\sum_{n=0}^{N-1} (2n+1)^2 = \frac{1}{3}N(4N^2 - 1) \tag{1}$$

A. Cube Labels

The cubes have labels on each of their faces, and importantly, cubes always have the same label on opposite faces. In a pyramid with N layers there are $2N - 1$ distinct labels. Each cube has a unique labeling (a, b, c) which is generated by choosing 3 labels from the set of $2N-1$ *with replacement*. The number of possible combinations with replacement is identical to the number of cubes:

$$\begin{aligned} \left(\!\binom{2N-1}{3}\!\right) &= \binom{(2N-1)+3-1}{3} \\ &= \frac{1}{3}N(2N-1)(2N+1) \\ &= \frac{1}{3}N(4N^2-1) \end{aligned} \tag{2}$$

Permutations of a label tuple (a, b, c) (e.g. (c, a, b)) correspond to spatial rotations of a cube and are not separate cubes. Cubes have up to 3 distinct labels and 6 possible rotations. Cubes with repeated labels have fewer distinguishable rotations. Each cube is unique and must be used exactly once to solve the pyramid. Cubes can be placed into the pyramid with any rotation.

B. Axis Labels

Due to the rule that opposite cube faces have identical labels, any line of cubes along an axis must have the same label all the way down the axis. Therefore one does not need to see the hidden faces to determine the state of the pyramid. It is only necessary to see the labels on the top, front, and right external faces.

A solution can be written in a compact ASCII-art format, suggested by Donald Knuth, that is reminiscent of an architect's elevation drawing. The solution for the same pyramid in Figure 1 is as follows:

```
+-----------------+
|  2  1  1  4  3  |  1
|  2  4  1  2  4  |  4  2
|  2  3  1  0  0  |  0  3  1
|  3  4  2  3  0  |  3  3
|  2  0  0  3  1  |  2
+-----------------+
   2  4  0  4  0
      1  1  3
         1
```

The labels on three orthogonal external faces determine the specific cube and rotation at the intersection point of the axes through those faces.

II. SAT Encoding

A solver implemented with Z3Py [2] is available at [1]. The constraint formulation uses bounded integer variables (or

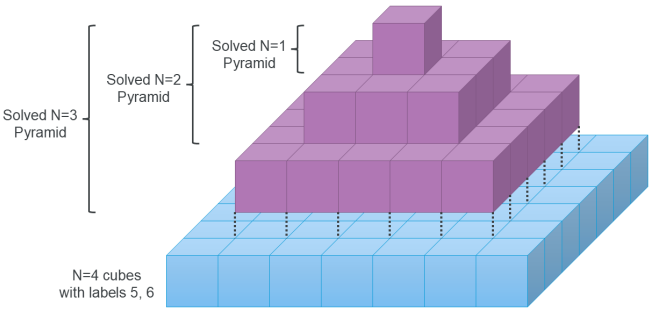


Fig. 2. Constructive Bottom Strategy

optionally enumeration sorts) for the axis label assignments, and Boolean variables to indicate that a specific cube is placed at a certain coordinate with a certain rotation. There are only two types of constraints: exactly-1 cardinality constraints assert that each cube must be placed at one coordinate with one rotation. The second type of constraint asserts that a conjunction of three orthogonal axis assignments is equivalent to one of the Boolean placement variables.

The Python program is capable of exporting the problem in CNF or SMT2 format. It can also re-import certificates to print and verify solutions generated by an external solver. The program has options documented on the GitHub page for applying different Z3 solver tactics and symmetry-breaking strategies.

III. Constructive Solutions

An algorithmic process for constructing solutions for any pyramid size N is not currently known. It is tempting to seek a recursive process that uses the solution for an $N-1$ pyramid to solve the N pyramid quickly. Two such proposals are given below, and SAT can be used to easily show that they do not generalize.

A. Adding a New Base Layer

The "new base layer" approach (Fig. 2) takes a solved pyramid with $N-1$ layers and adds the next layer of size $(2N-1) \times (2N-1)$ underneath it. An N-layer pyramid has two additional labels compared to an $N-1$ pyramid. All of the cubes with the two new labels must be located on the new bottom layer. The same property applies for each layer up to the top of the pyramid, which must be the trivial $N=1$ subproblem with the single cube $(0,0,0)$. This recursive approach works only for N up to 3 and is UNSAT for $N=4$, proving that this strategy cannot be the basis of a general solution.

B. Adding a New Top Shell

This approach takes a solved $N-1$ pyramid and drops the next layer of cubes on top of it to create an N-layer pyramid. The trivial $(0,0,0)$ pyramid is at the center of the bottom layer. This approach also only works for N up to 3.

IV. Symmetry-Breaking Strategies

The puzzle has various geometric symmetries and is also symmetric under permutations of the labels. The solver program has options to apply additional constraints to break some of these symmetries.

- The label permutation symmetry can be broken by adding lexicographical ordering constraints on the axis label variables. The axis labels can be constrained left-right, top-down as one would read the solution. Heuristic importance can also be considered. Labels on the longest axes control the most number of cubes and may hypothetically benefit from more constrained assignments to limit the search space.
- Mirroring and rotation geometric symmetries can be broken by applying ordering constraints between pairs of axis label variables on the front and right faces of the pyramid. There is an additional geometric symmetry that allows entire vertical slices of the pyramid on opposite sides of the center line to be swapped without affecting any other cubes.
- A few types of look-ahead constraints are considered to prevent the search from exploring subspaces where too many axes have been assigned to the same label, such that there aren't enough blocks with that label to fill all of the spaces along those axes.
- In addition to the infeasible recursive constructive strategies described earlier, there are also options for *partial* constructive strategies that don't build on independently solved subproblems. These strategies place or constrain some blocks, or assign or constrain some axis labels, and then use SAT to try to solve the rest of the pyramid. Often they are based on intuition or informal geometric patterns in the hope of yielding insight towards a general solution. None have yet been successful.

V. Results

$N=4$ pyramids can be solved by Kissat [3] in less than a minute. $N=5$ pyramids with partial constructive strategies to limit the search space take on the order of ten minutes. Even with the fastest known combination of strategies, no $N=6$ pyramid has yet been solved. Up to a week of run time using ParKissat-RS on a 32-thread machine has been attempted.

The proposed benchmarks use combinations of pyramid sizes and symmetry-breaking strategies to provide variations in difficulty in the 2–60 minute range. UNSAT instances that disprove a selection of recursive constructive strategies are also included. Mutually-incompatible combinations of symmetry strategies also provide interesting UNSAT instances.

References

[1] A. Mihal. "Bel's Pyramid Puzzle," 2025, GitHub repository. [Online]. Available: https://github.com/acmihal/bel_pyramid
[2] L. De Moura and N. Bjørner. "Z3: An Efficient SMT Solver," In TACAS'08. 337–340
[3] A. Biere et al. "CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2024". In Proc. SAT Comp. 2024. 8–10

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

CNF Generation of Miter Circuits

Yuhang Qian[1, 2], Zhihan Chen[1, 2], Xindi Zhang[1, 2], Shaowei Cai[1, 2, *]
[1]Key Laboratory of System Software (Chinese Academy of Sciences) and State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences
[2]School of Computer Science and Technology, University of Chinese Academy of Sciences
{qianyh, chenzh, zhangxd, caisw}@ios.ac.cn

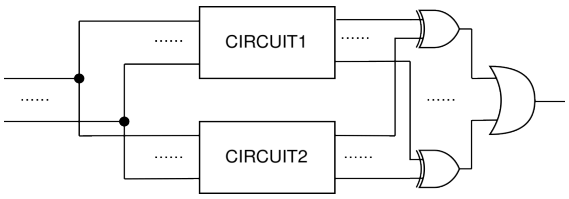


Fig. 1. Miter Circuit

Introduction

The problem of logical equivalence checking (LEC) remains a central challenge in hardware verification, especially for structurally complex benchmark circuits. In recent years, the SAT-SWEEPING framework [3] has become a mainstream method for equivalence checking, leveraging SAT solvers to efficiently validate circuit correctness. However, its effectiveness is closely tied to the performance of the underlying SAT solver. To support research on solver efficiency and robustness, this submission provides a collection of miter circuits constructed from standard academic benchmarks, including ISCAS85, ISCAS89, and ITC99, for the SAT Competition 2025.

Miter Circuit

Miter circuit [2] is a logic circuit that compares the outputs of the two circuits being verified. Fig. 1 is an example. Specifically, the construction of the miter circuit involves connecting the inputs of two circuits whose equivalence is to be checked, with both circuits sharing the same input. The corresponding outputs are then connected via XOR gates, and all of these XOR gates are taken as inputs to an OR gate, with the output of the OR gate serving as the output of the miter circuit. If there is a set of inputs for which the output of the miter circuit is 1, it indicates that the circuits are not equivalent. Conversely, if such inputs do not exist, the two circuits are equivalent.

Generation of Benchmarks

We use academic miter circuits which are generated from the ISCAS85, ISCAS89, and ITC99 benchmark suites, emphasizing circuits of moderate complexity for demonstration. The method for generating miter circuits involves first converting the circuit into a combinational circuit using scan chains. Then, logic synthesis is performed using the ABC tool [1] before creating the miter circuit. The logic synthesis commands [4], &st; &dch; &if -K 6; &mfs;, are executed three times iteratively.

Converting AIG format to CNF format is very easy. Only the encoding of AND gates needs to be considered, for example $c = a\ And\ b$ can be encoded to CNF $(\neg c \vee a) \wedge (\neg c \vee b) \wedge (c \vee \neg b \vee \neg a)$ As for the NOT logic in AIG, it can be achieved by inverting the symbol.

Benchmarks

We removed instances that were either too easy or too hard, and selected 20 examples of moderate difficulty. The benchmarks are shown in Table I.

TABLE I
Miter Circuit Benchmark

benchmark	num_vars	num_clauses
b14.cnf	12308	34638
b15.cnf	18696	51951
b17.cnf	58528	162579
b18.cnf	167755	473169
b18_1.cnf	163609	460737
b19.cnf	337330	952029
b19_1.cnf	329172	927579
b20.cnf	25775	72726
b20_1.cnf	21782	60747
b21.cnf	26076	73629
b21_1.cnf	21802	60807
b22.cnf	38666	109200
b22_1.cnf	33427	93483
c3540.cnf	2163	6216
c5315.cnf	3801	10401
c7552.cnf	4004	11076
s15850.cnf	8534	21297
s35932.cnf	21095	54351
s38417.cnf	20100	50412
s38584.cnf	28044	71346

References

[1] ABC. https://github.com/berkeley-abc/abc.
[2] D. Brand. Verification of large synthesized designs. In Proceedings of 1993 International Conference on Computer Aided Design (ICCAD), pages 534–537. IEEE, 1993.
[3] A. Mishchenko, S. Chatterjee, R. Brayton, and N. Een. Improvements to combinational equivalence checking. In Proceedings of the 2006 IEEE/ACM international conference on Computer-aided design, pages 836–843, 2006.

[4] H.-T. Zhang, J.-H. R. Jiang, L. Amarú, A. Mishchenko, and R. Brayton. Deep integration of circuit simulator and sat solver. In 2021 58th ACM/IEEE Design Automation Conference (DAC), pages 877–882. IEEE, 2021.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Round-Robin Tournament Scheduling Benchmarks

Joseph E. Reeves, Aeacus Sheng
Carnegie Mellon University
Pittsburgh, United States

***Abstract*—We present a family of unsatisfiable benchmarks that arise from round-robin tournament scheduling. The family has two classes: a classical formulation and a multi-venue variant that restricts the number of rounds that can be played each day. These instances examine the ability of SAT solvers to reason about exactly-one cardinality constraints.**

I. Problem

Let n be a number of teams and D be a number of match days. Every unordered pair of teams must meet exactly once. We distinguish two variants.

Classical round-robin. A match is assigned to a day. Any number of different matches may take place on the same day, provided no team is scheduled twice that day.

Multi-venue round-robin. Each day offers V venues. A match occupies one *(day, venue)* slot. Besides the team/day restriction, no venue may host two different matches on the same day.

II. Encodings

Indices satisfy $1 \leq i < j \leq n$ for teams, $1 \leq d \leq D$ for days, and $1 \leq v \leq V$ for venues (with $V = 1$ in the classical case). For the exactly one constraint we use the direct encoding, using one long clause and a pairwise at-most-one encoding (introducing a quadratic number of binary clauses in the number of literals).

A. Classical Variant

Variables: $x_{i,j,d} = 1$ iff teams i and j play on day d.

Exactly-one per pair. For each pair of teams $i < j$:

1) one **at-least-one** clause $\left(x_{i,j,1} \vee \cdots \vee x_{i,j,d}\right)$;
2) **binary at-most-one** clauses $(\neg x_{i,j,d_1} \vee \neg x_{i,j,d_2})$ for all $d_1 < d_2$.

Daily team capacity. For each team p, day d, and teams $q_1, q_2 > p$:

$$(\neg x_{p,q_1,d} \vee \neg x_{p,q_2,d}), \quad q_1 \neq q_2.$$

B. Multi-Venue Variant

Variables: $x_{i,j,d,v} = 1$ iff the game between teams i and j $(1 \leq i < j \leq n)$ is scheduled on day d $(1 \leq d \leq D)$ in venue v $(1 \leq v \leq V)$.

Exactly-one per pair: For every pair $i < j$:

1) one **at-least-one** clause $\left(x_{i,j,1,1} \vee \cdots \vee x_{i,j,D,V}\right)$;
2) **binary at-most-one** clauses $\left(\neg x_{i,j,d_1,v_1} \vee \neg x_{i,j,d_2,v_2}\right)$ for all distinct slots $(d_1, v_1) \neq (d_2, v_2)$.

Note that in the multi-venue case, daily team capacity is implied in the binary clauses above.

Venue capacity: A venue host at most one match per day. For every day d, venue v and distinct matches $(i, j) \neq (p, q)$:

$$(\neg x_{i,j,d,v} \ \vee \ \neg x_{p,q,d,v}).$$

III. Satisfiability

A given instance will be satisfiable if and only if the schedule parameters (n, D, V) allow all $\binom{n}{2}$ pairwise games to be assigned without violating the above constraints. In practical terms, unsatisfiability arises when the number of days D is below the minimum required to schedule all matches for n teams with V venues. Each team must play $n - 1$ games in total, at a rate of at most one game per day, which alone implies $D \geq n - 1$ (for even T) or $n \geq T$ (for odd T)[1]. Moreover, since at most V games can take place in parallel on any day, a basic capacity constraint is $D \cdot V \geq \binom{n}{2}$ (the total number of games).

IV. Benchmarks

The satisfiable round-robin problems are usually solved by CaDiCaL in under a second, so only the unsatisfiable instances are interesting. We produced 10 classical and 10 multi-venue benchmarks, all of which are unsatisfiable because $D < n - 1$. Together with generators, they can be found at `https://github.com/Aeacu2/RR-Benchmarks`. All benchmark filenames encode their key parameters. Classical (unlimited-venue) instances are named `RoundRobin_n<n>_d<D>.cnf`, where `<n>` denotes the number of teams and `<D>` the number of days. Multi-venue instances are named `MVRoundRobin_n<n>_d<D>_v<V>.cnf`, with the additional field `<V>` recording the number of venues available per day.

V. Comparison

Adding venue constraints can make the problems significantly more difficult for solvers. On a 14-core Intel Core i7-1280P (2.00 - 4.80 GHz) processor with 16GB Memory, `RoundRobin_n12_d10.cnf` can be solved in under 30 seconds, while `MVRoundRobin_n12_d10_v2.cnf` takes more than 3600 seconds.

[1] In a round-robin with an odd number of teams, one team is idle each day, so n days are needed for each team to complete $n - 1$ games. If n is even, it is possible to schedule all games in $n - 1$ days.

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

SAT Encodings of Mechanical Master-Key Systems

Niccolò Rigi-Luperti
Institute of Theoretical Informatics
Karlsruhe Institute of Technology
Karlsruhe, Germany
niccolo.rigi-luperti@kit.edu

Dominik Schreiber
Institute of Theoretical Informatics
Karlsruhe Institute of Technology
Karlsruhe, Germany
dominik.schreiber@kit.edu

***Abstract*—We present a benchmark set submitted to the SAT Competition 2025, consisting of SAT encodings of the Master-Key System problem. This submission is of rather archival nature: The encodings originate from Radomír Černoch's doctoral thesis (2017), but have never found their way into any public benchmark set – we remedy this here.**

I. Introduction

The NP-hard [1], [3] Master-Key System problem (MKS) describes a situation where for a set of mechanical keys and locks it is specified which key should open which lock, but the locks and keys are manufactured under constraints. A real-world instance of this problem occurs, for example, when keys have to be produced for a large building complex and different users groups demand different access-hierarchies.

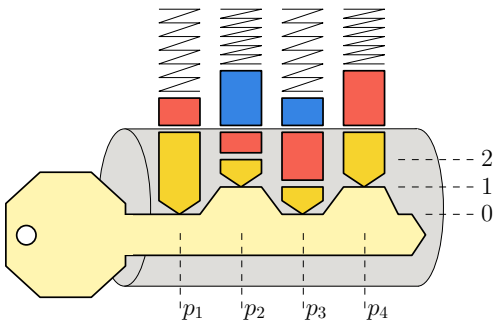


Fig. 1. A key $k = (0, 1, 0, 1)$ and a lock $l = ((0), (1, 2), (0, 2), (1))$. The key can open the lock because it pushes the pin-column gaps into the shear line (at the border of the cylinder). A second key like $k_2 = (0, 1, 2, 1)$ could also open the lock, it would push the third column higher to it's second gap. A key like $k_3 = (1, 1, 0, 1)$, however, could not open the lock because it would push the first column into the shear line, blocking rotation of the cylinder.

The following description is heavily based on the doctoral thesis of Radomír Černoch [1] and the master's thesis of Martin Hořeňovský [2]. In 2017, Černoch presented the first extensive theoretical and practical treatment of encoding the MKS problem into SAT, also showing a P to NP transition between different MKS variants. With off-the-shelf SAT solvers and additional optimizations, the approach reportedly outperformed proprietary software of a collaborating company. However, to our knowledge, no benchmarks or CNF files of this work have ever been published. We see our contribution thus in resurfacing this distinct SAT instance family, presenting the basic encoding, re-implementing it [4], and adding its first CNF benchmarks to the public database.

II. Lock-Charts

A mapping between locks and keys is represented in a lock-chart, with one row per lock and one column per key. When a key should open a lock, the respective cell is marked black.

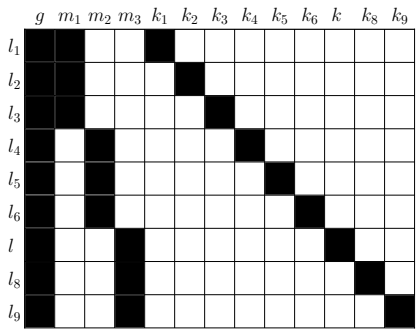


Fig. 2. A lock-chart with $L = 9$ locks and $K = 13$ keys. The keys are of three different types: One general key g, three master keys m_i, and nine single keys k_i. Black cells indicate that the key can open the lock.

To model access-hierarchies, five key types are introduced:

- A general key g, which opens all locks.
- Three master keys m_i that each open a third of all locks.
- Single keys k_i that each open a single lock.
- Ascending keys, which each open one more lock than the previous ascending key.
- Triple keys, which each open three locks.

A small chart with only the first three key types is given in Fig. 2. A larger general lock-chart with all five key types is shown in Fig. 3. We create structured synthetic lock-charts by choosing a number of locks L and then filling the chart with all five key types. For algorithmic comparison we also create random lock-charts. Given L, we insert $K = 2L$ keys and let each key open each lock with probability p.

III. Encoding

Keys have P positions and at each position they are cut to one of D depths. In Fig. 1, a key with $P = 4$ positions and $D = 3$ available depths is shown. Locks have also P positions, but they can accept multiple depths per position. This is achieved trough pin columns which create multiple gaps; it suffices that a key pushes any one of the gaps into the shear line of the cylinder.

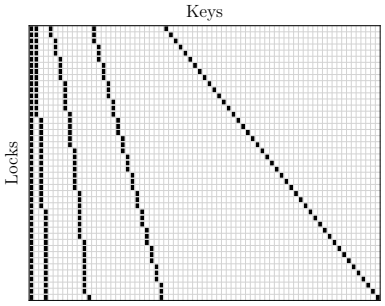


Fig. 3. A larger lock-chart with $L = 45$ locks, leading to $K = 73$ keys. Here all five key types are included. The ascending keys can be recognized by their 2,3,4,5... pattern.

The geometry of keys and locks is encoded via variables $key^k_{p,d}$ and $lock^l_{p,d}$, respectively. Variable $key^k_{p,d}$ is true exactly when key $k \in K$ at position $p \in P$ has a depth of $d \in D$. Similarly, variable $lock^l_{p,d}$ is true exactly when lock $l \in L$ at position p has pins such that depth d pushes a gap into the shear line, i.e. the lock "allows" the cut (p, d).

Each key k must have at least one cut depth d at every position p.

$$\forall k, p: \quad \bigvee_d key^k_{p,d} \tag{1}$$

For each key there can be at most one depth d per position.

$$\forall k, p, d \neq d': \quad \neg key^k_{p,d} \vee \neg key^k_{p,d'} \tag{2}$$

For this constraint binary encoding suffices, as the maximum cut depth is $D \leq 4$ in all our benchmarks. For brevity we write $k \in K$ for $k \in [1, ..., K]$, and when it is clear that we iterate over a whole set we also omit the set and just write $\forall k$, analog for all other indices.

A lock l must be open for its keys $k \in \mathcal{K}(l)$, i.e., it must allow every cut (p, d) that one of its keys has.

$$\forall l, p, d: \quad \left(\bigvee_{k \in \mathcal{K}(l)} key^k_{p,d} \right) \Leftrightarrow lock^l_{p,d} \tag{3}$$

Technically an implication from left to right would suffice, but an equivalence prohibits any superfluous pins that would make the lock unnecessarily permissive.

To model blocking, a helper variable $block^{l,k}_{p,d}$ is introduced. When it is true, it implies that key k cannot open lock l due to they key's cut of depth d at position p.

$$\forall l, k, p, d: \quad block^{l,k}_{p,d} \Rightarrow key^k_{p,d} \wedge \neg lock^l_{p,d} \tag{4}$$

For every lock l and each of its blocked keys $k \notin \mathcal{K}(l)$, this blocking is enforced by requiring that at least one of the blocking helper variables is true for some position and some depth.

$$\forall l \; \forall k \notin \mathcal{K}(l): \quad \bigvee_{p,d} block^{l,k}_{p,d} \tag{5}$$

Last, most keys don't have abrupt changes in their depth, as such cuts could easily break and are harder to manufacture. Thus we model a "smoothed" key profile by restricting the maximal jump in depth to ± 1 between neighbouring positions.

$$\forall k, p, d, d': |d - d'| > 1: \quad \neg key^k_{p,d} \vee \neg key^k_{p+1,d'} \tag{6}$$

IV. Benchmarks

We created three benchmark groups. The first two groups use key geometries $P = 8$ and $D = 4$. Accounting for the jump restriction, this allowed for ca. $3.5^8 \approx 22\,000$ key geometries for ca. $K = 200...400$ lock-chart keys.

For the first benchmark group we chose $L \in [190, 200, 210, 220, 230, 235, 240, 245, 250, 255]$ locks and created for each L its unique synthetically structured lock-chart. On test runs on two CPUs with Kissat, $L = 190$ ran in ca. 1300 s and $L = 220$ in ca. 4200 s (all satisfiable). We expect thus in particular the instances beyond $L = 220$ to be challenging but still satisfiable.

For the second group, we tried much smaller but randomized lock-charts, with L from 18 to 21. The probability for lock opening was set to $p = 0.3$. Run times varied by multiple orders of magnitude, from minutes to hours. We chose one instance which eventually finished in 1700 s ($L = 18$) and one that finished in 5800 s ($L = 19$), as well as three others (all also $L = 19$) that did not finish within two hours. The two that finished where both satisfiable.

For the third and last group, we wanted to find unsatisfiable instances. We shrunk the key universe drastically to $P = 4$ and $D = 3$, i.e., to only ca. $2.5^4 \approx 40$ unique key geometries, and chose a structured lock-chart. For $L = 10$ (leading to $K = 22$), satisfiable solutions were found immediately. For $L = 11$ (giving $K = 23$), however, they were proven unsatisfiable after ca. 2800 s. We chose $L \in [11, 12, 13, 14, 15]$ as the benchmark instances, all with similar run times within 2000-3000 s, and all five are unsatisfiable.

The submitted files all start with `lockchart-group`n, with $n \in \{1, 2, 3\}$ for the respective group.

Group	#Inst.	Typ	Key geom.	Lock number L	Lock-chart
1	10	SAT	P=8, D=4	$\in [190, ..., 255]$	structured
2	5	SAT (?)	P=8, D=4	$\in [18, 19]$	randomized, p=0.3
3	5	UNSAT	P=4, D=3	$\in [11, ..., 15]$	structured

TABLE I
THE SUBMITTED BENCHMARK INSTANCES

Group	#Variables	#Clauses	Ratio	Size (non-compressed)
1	1.7 – 3.0 Mio	3.4 – 6.0 Mio	≈ 2.0	60 – 116 MB
2	22 k – 24 k	39 k – 44 k	≈ 2.0	600 – 700 kB
3	3.4 k – 5.7 k	6.8 k – 10.8 k	≈ 2.0	80 – 140 kB

TABLE II
MORE INSTANCE DETAILS

References

[1] Radomír Černoch, "*Lock-chart solving*", PhD Thesis, Czech Technical University in Prague, 2017. URI: http://hdl.handle.net/10467/73582
[2] Martin Hořeňovský, "*Performance analysis of a master-key system solver*", MA Thesis, Czech Technical University in Prague, 2018. URI: http://hdl.handle.net/10467/73952
[3] Anna Lawer, "*Calculation of Lock Systems*", MA thesis, Royal Institute of Technology, 2004.
[4] Lock-chart to CNF converter: https://github.com/nrilu/lockchart-to-cnf/

Proceedings of SAT Competition 2025: Solver and Benchmark Descriptions, Proceedings of SAT Competitions, Vienna, 2025.
This work is licensed under a Creative Commons "Attribution 4.0 International" license.

Benchmark Description for dyCaDiCaL Submission to SAT Competition 2025

Junjie Xu
Ningbo University
Ningbo, China
xjj375227926@gmail.com

***Abstract*—This document describes a benchmark set submitted for the Main Track of the SAT Competition 2025. The benchmark set contains 20 previously unpublished CNF instances with varying difficulty and structure, designed to evaluate the performance of CDCL-based SAT solvers. The selected problems cover a wide range of scales, from small to industrial-level instances, to test solver robustness across different problem sizes.**

I. Benchmark Overview

The submitted benchmark instances are in standard DIMACS CNF format. Each file starts with optional comments followed by a `p cnf` line and a list of clauses. All instances have been verified for correct formatting and satisfiability using standard SAT tools. Table I summarizes the variable and clause counts for each case.

TABLE I
Benchmark Case Statistics

Case	#Variables	#Clauses
Case1	21888	253852
Case2	2200	9085
Case3	26390	316875
Case4	1002	113964
Case5	14756	141682
Case6	600	5401
Case7	600	5398
Case8	15222	18687
Case9	500	4502
Case10	600	5405
Case11	760	43648
Case12	450	4045
Case13	14174	66783
Case14	3008	9983
Case15	30587	91702
Case16	600	5404
Case17	550	4949
Case18	245249	1089830
Case19	138711	4688613
Case20	945	61408

II. Difficulty and Diversity

The difficulty of the benchmarks was estimated based on solver runtime. At least 10 of the instances are of medium complexity—neither trivially solvable within one minute by MiniSat, nor unsolvable within one hour by dyCaDiCaL. The set includes small handcrafted examples and larger industrial-scale instances, designed to reflect both practical and theoretical SAT challenges.

III. Format Compliance

All CNF files comply with the SAT Competition 2025 submission guidelines:

- No clause contains complementary literals.
- Variable numbers are within declared bounds.
- All clauses terminate with a '0' as required.

IV. Acknowledgment

This benchmark set was prepared as part of the dyCaDiCaL solver development project for the SAT Competition 2025.

Solver Index

AE_kissat2025_bump, 15
AE_kissat2025_MAB, 15
AE_kissat2025_rescale, 15
amSAT, 30

CaDiCaL, 10
CaDiCaL-Alignable-Independent-ULC, 22
CaDiCaL-Alignable-ULC, 22
CaDiCaL-Alignable-ULC-order, 22
CaDiCaL-XLC, 22

dilemma, 26
DynamicCaDiCaL, 28

Gimsatul, 10

hCad-psbva, 27
hCad-sbva, 27
hKis-sbva, 27
hKis2-psids, 27

IsaSAT, 10

Kissat, 10
Kissat-CURE, 11
Kissat-Pred, 20
Kissat-Pred-Aggressive, 20
Kissat_CoRephase_CoReward, 13
Kissat_MAB_CoRephase, 13
Kissat_Scavel2025, 19
Kissat_Scavel_RS, 19
Kissat_VSA, 18

MallobSat, 25

PL-PRS-GASPI-KISSAT, 24
PL-PRS-KISSAT, 24
PRS-SC25, 21
PRS-SC25-SBVA, 21

Satsuma+Kissat, 8

Benchmark Index

Combinational Equivalence Checking, 42

Equivalence Checking of Hardware Multiplier Architectures, 35
Equivalence Checking of MD5 Circuits, 37

Hard Combinatorial Instances with Symmetry, 33

Mechanical Master-Key Systems, 50
Miter Circuits, 47
Model Checking, 38

Odd Ball Weighing Problems, 43

P-Center Problem, 39
Preferred Extension of Argument Framework, 40
Pyramid Puzzle, 45

Round-Robin Tournament Scheduling, 49

SC24 Benchmarks Adjusted, 52
Split-BDD Multiplier Verification, 34

Author Index

Anders, Markus, 8, 33

Baarir, Souheib, 24
Biere, Armin, 10, 25, 34, 38
Bogaerts, Bart, 8, 33
Bogø, Benjamin, 8, 33

Cai, Shaowei, 21, 47
Cai, Yalun, 11, 35
Chen, Li, 19
Chen, Runyao, 15
Chen, Shuang, 13, 37
Chen, Shuwei, 19
Chen, Zhihan, 21, 47
Cui, Shaoke, 20, 42

Ding, Hang, 15
Djamegni, Clémentin Tayou, 27

Faller, Tobias, 10
Fleury, Mathias, 10
Froleyks, Nils, 10, 38

Gontier, Arthur, 8, 33

Haba, Belgacem (Bel), 45
Hu, Chunming, 20, 42
Huang, Yiwen, 30

Kaufmann, Daniela, 34
Konrad, Alexander, 34
Koops, Wietze, 8, 33

Lejemble, Thibault, 24
Li, Chu-Min, 15, 30, 40
Li, Kezhi, 11, 35
Li, Shunwei, 15
Li, Shuolin, 18, 30, 39
Li, Tiejun, 13
Li, Zhihui, 19
Liu, Jinjin, 13
Liu, Qi, 40
Luo, Chuan, 20, 42
Luo, Mao, 15, 30, 40
Lü, Zhipeng, 30

Mihal, Andrew, 43, 45

Nordström, Jakob, 8, 33

Oertel, Andy, 8, 33

Pollitt, Florian, 10

Qian, Yuhang, 21, 47

Reeves, Joseph E., 22, 49
Rigi-Luperti, Niccolò, 25, 50

Saoudi, Mazigh, 24
Saouli, Sabrine, 24
Scholl, Christoph, 34
Schreiber, Dominik, 25, 50
Sheng, Aeacus, 22, 49
Shi, Zhengyuan, 11, 35
Sopena, Julien, 24
Sun, Yan, 13, 37

Takacs, Benjamin, 26
Tao, Mengxia, 11, 35
Tchinda, Rodrigue Konan, 27

Wu, Guanfeng, 19
Wu, Xinyun, 15

Xiong, Caiquan, 15, 40
Xu, Junjie, 28, 52
Xu, Qiang, 11, 35
Xu, Yang, 19

Yang, Zhenwei, 20, 42

Yu, Emily, 38

Zeng, Chuang, 30
Zhang, Jialu, 18, 39
Zhang, Jianmin, 13, 37
Zhang, Xindi, 21, 47
Zhu, Jiaying, 11, 35